Конфіденційно

Vision — аудит безпеки

Повторна перевірка після виправлень, перед перенесенням у банківський контур

Коміт	`a3b6104` (гілка `main`, після PR #33); попередній аудит — `7b13af1`
Дата	7 листопада 2026
Скоуп	монорепо `nazarenkod/vision`: backend, prisma, frontend, CLI/MCP, sdk-java, scripts, infra, CI
Модель розгортання	внутрішній on-prem сервіс; кілька команд в одному інстансі; на тій самій машині, що й робочий VRT
Промт	`audit/REVIEW-PROMPT.md` (без змін); попередній звіт — `audit/report-2026-11-05.pdf`
Класифікація	Конфіденційно — лише для керівництва, ІБ та розробників

Зміст

1. Резюме для керівництва
2. Скоуп, методологія та обмеження
3. Архітектура та модель загроз
4. Регресія: знахідки аудиту 2026-11-05 і S-1…S-15
5. Зведена таблиця знахідок
6. Детальні знахідки
7. Дані за межами контуру та LLM
8. Комерційне використання та ліцензії
9. Roadmap виправлень
10. Додатки

1. Резюме для керівництва

Це повторна перевірка Vision після виправлень за аудитом 5 листопада 2026 (41 знахідка). Мета — переконатися, що виправлення справді працюють, немає обходів, а нові зміни (PR #23–#33) не внесли нових вад. Ключові виправлення перевірено **наживо на локальному стенді** із синтетичними даними. Юніт-тести бекенду проходять повністю (818 із 818).

Загальний рівень ризику: СЕРЕДНІЙ-ВИСОКИЙ (було ВИСОКИЙ). Критичну вразливість закрито, і це відтворено на стенді: ескалації editor→admin і переносу даних між командами більше немає. Закрито або суттєво пом'якшено п'ять із шести високих: DoS аналізом доступності, зависання воркера, захоплення акаунта LDAP, порожній пароль LDAP. VLM на Gemini власник прийняв як ризик, бо з Gemini є контракт. Проте лишилися **дві нові високі вади**, які попередній аудит не бачив, і **обхід виправлення ліміту знімків**, підтверджений наживо.

High 4 **Medium 18** **Low 27** **Info 5** **Закрито 3**

Відкритих знахідок — 54: 39 із попереднього аудиту і 15 нових. Critical — 0.

Що змінилось від 2026-11-05.

- **Закрито повністю (3):**
  - VIS-SEC-001 — масове присвоєння, Critical;
  - VIS-SEC-007 — порожній пароль LDAP;
  - VIS-SEC-012 — `StripUnknownFieldsPipe` на всіх маршрутах.
- **Виправлено частково (5):**
  - VIS-SEC-004 — DoS аналізом доступності, знижено до Medium;
  - VIS-SEC-005 — області ігнорування, знижено до Medium;
  - VIS-SEC-006 — LDAP-email, знижено до Low;
  - VIS-SEC-014 — ліміт знімка: є обхід, див. VIS-SEC-045;
  - VIS-SEC-036.
- **Прийнятий ризик:** VIS-SEC-003. Рішення власника 2026-11-07: контракт із Gemini.
- **Відкладено:** VIS-SEC-002, рішення власника.
- **Без змін (30):**
  - **з них Medium (13):** VIS-SEC-008, 009, 010, 011, 013, 015, 016, 017, 018, 019, 020, 021, а також 035, яку піднято з Low до Medium.
  - Зверніть увагу: у журналі виправлень (`audit/PROGRESS.md`) з Medium позначено лише 012 і 014. Решта Medium **не виправлялись**, і ця перевірка це підтверджує.
- **Нові (15):** 2 High, 2 Medium, 9 Low, 2 Info. Частина — наслідки самих виправлень (VIS-SEC-044, 045) або нових екранів #23–#31 (VIS-SEC-043, 048).

Топ-5 ризиків зараз:

1. **Відкриття одного знімка блокує API для всіх команд** (VIS-SEC-043, High, **відтворено**).
   - Дерево екрана з 10 000 безіменних елементів важить 110 КБ у запиті.
   - `GET /test-runs/:id/structure` для нього рахується 6,8 с, і стільки ж чекають усі інші запити до API.
   - Складність квадратична: 30 000 елементів — близько хвилини на кожне відкриття.

2. **Локальний пароль обходить LDAP/SSO банку** (VIS-SEC-044, High).
    - У режимі SSO форма пароля приймає паролі, скопійовані з робочого VRT.
    - У режимі LDAP людина, яку прибрали з каталогу, після виправлення 41b15e0 входить старим локальним паролем.
    - Відкликання доступу через каталог чи IdP не спрацьовує.
3. **Обхід ліміту розміру знімка** (VIS-SEC-045, Medium, **відтворено**).
    - PNG на 560 КБ із другим заголовком IHDR проходить перевірку.
    - Під час порівняння розпаковується в 144 Мпкс: процес API виріс до 2,4 ГБ.
    - Більший файл покладе процес.
4. **Слабкі або спільні з продом секрети підпису** (VIS-SEC-002, High, відкладено). `change-me` приймається, а DEPLOY.md і далі радить секрет робочого VRT.
5. **Немає CSP/HSTS і TLS у поставці, немає відкликання сесій** (VIS-SEC-011, 009, Medium). Разом із JWT у localStorage (S-13) одна XSS дає добу повного доступу без можливості погасити токен.

Вердикт готовності до банківського контуру: УМОВНО НЕ ГОТОВО.

- **Блокери, обидва невеликі за обсягом:**
    - VIS-SEC-043 — квадратичний `/structure`: обмежити кількість вузлів або винести у воркер;
    - VIS-SEC-045 — перевіряти всі IHDR або розміри після декодування в заголовку pngjs.
- **Потрібне рішення власника:** VIS-SEC-044 і VIS-SEC-002 (бракує позначки походження акаунта та перевірки сили секретів).
- Після цього достатньо короткої повторної перевірки цих пунктів.
- Medium VIS-SEC-008, 009, 011, 013, 018, 020 — у план на 30 днів. Для банку їх варто закрити до промислового використання з реальними даними.

2. Скоуп, методологія та обмеження

Предмет. Дерево `main` на коміті `a3b6104`: виправлення 3d18853, 88669e6, 41b15e0, d3e852f, f52879b, e54fc5b і злиті UI-зміни #23–#31. Промт той самий, що й для аудиту 2026-11-05 (`audit/REVIEW-PROMPT.md`). Рішення власника з нього діють і далі:

- історію git не розглядаємо;
- передача коду компанії — внутрішнє використання;
- стенд нативний.

Робочий VRT, його база, контейнери й папка картинок не чіпались узагалі (правило 1). Скрипти `demo-clone-from-prod.sh`, `vision-backup.sh` і `vision-restore.sh` не запускались. Зовнішніх запитів у Jira, Gemini і Google Drive не було.

Методологія. Етапи ті самі: розвідка, сканування, ручний аудит і динамічне підтвердження.

- **Регресія.** Кожну з 41 знахідки звірено з кодом `a3b6104`: виправлення, тест, можливий обхід.
- **Ручний аудит.** Чотири паралельні напрями:
  - A — автентифікація й HTTP;
  - B — авторизація й ізоляція;
  - C — знімки, дерева й ввід;
  - D — зовнішні виклики, інфраструктура, ліцензії.
- **Перевірка знахідок агентів.** Кожну нову знахідку рівня High/Medium від агента перевіряв окремо:
  - **на стенді:** VIS-SEC-043, 045;
  - **заміром:** VIS-SEC-049 (знижено до Low);
  - **за кодом:** VIS-SEC-044, 047 (знижено до Low).

Інструменти:

Напрям	Інструмент	Результат запуску
Залежності	npm audit (prod і dev, 3 пакети)	виконано
Залежності	osv-scanner 1.9.1	**не виконано**: api.osv.dev недоступний (403 проксі)
SAST	semgrep 1.x, 969 правил із репозиторію semgrep-rules (javascript, typescript, python, java, dockerfile, bash, generic/secrets)	виконано; реєстр semgrep.dev недоступний
SAST	bandit; shellcheck 0.10.0	виконано
Секрети	gitleaks 8.21.2 (режим `dir`, без історії)	виконано
IaC	hadolint 2.15.1	виконано
IaC	trivy	**не виконано**: бінарник не отримано, використано результати 2026-11-05
Ліцензії	license-checker (backend prod)	виконано
SBOM	syft 1.18.1 (CycloneDX)	виконано

Напрям	Інструмент	Результат запуску
Тести	jest, юніт-тести бекенду (Node 24.22)	105 наборів, 818/818 зелені
Динаміка	нативний стенд: PostgreSQL 16 (окремий кластер, порт 55433) + API з `dist` `a3b6104`	лише синтетичні дані

Обмеження:

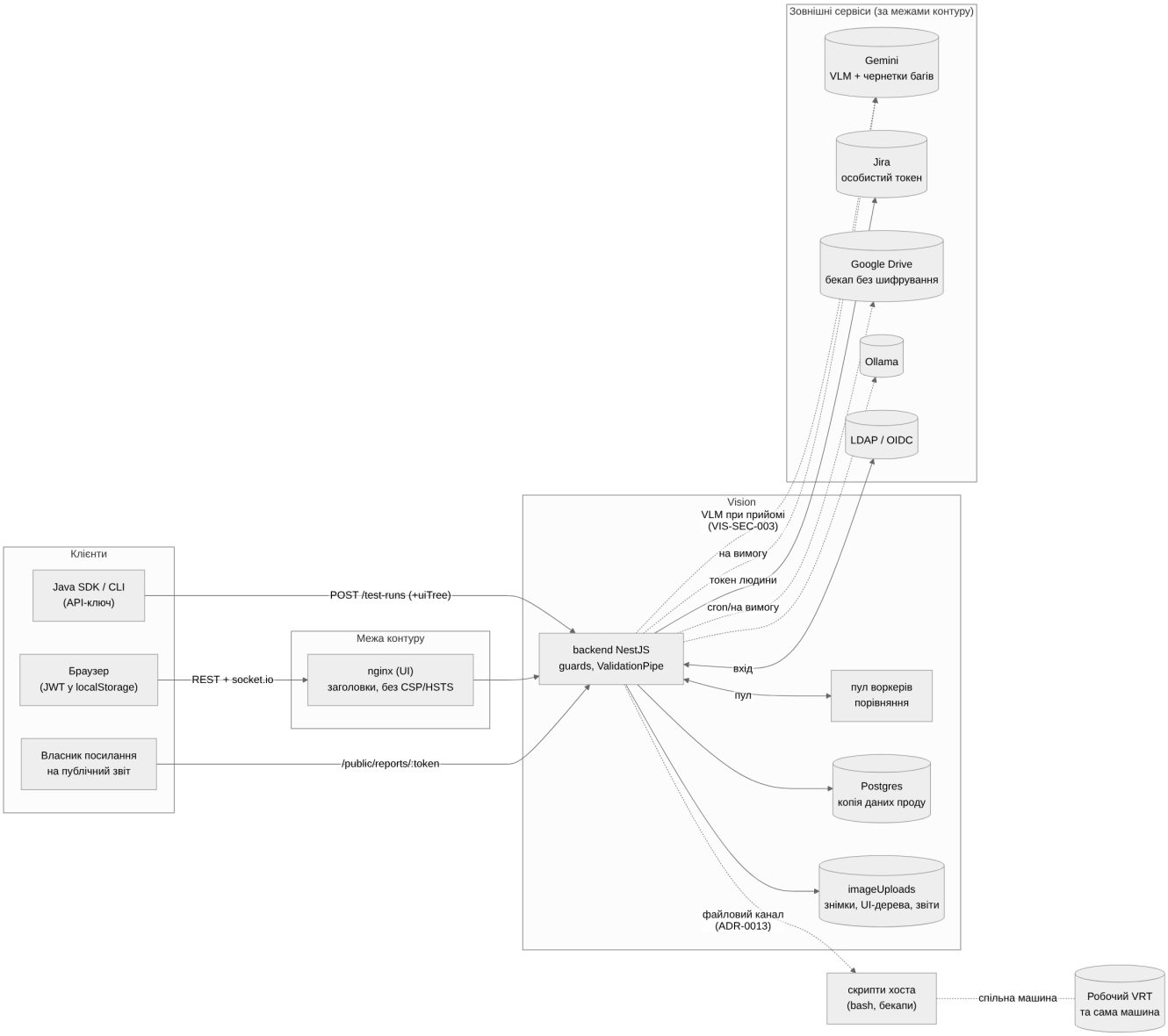
- **Trivy і OSV не виконувались.** Мережу заблоковано політикою середовища; повторне завантаження trivy з GitHub відхилив запобіжник середовища.
  - Покриття залежностей: `npm audit` (0) і результати trivy/osv від 2026-11-05.
  - Від тоді lock-файли змінились лише в межах злитих PR. Новий прогін `npm audit` дав 0 вразливостей.
- **Без `trivy image` і без Docker.** У середовищі немає демона Docker. Пакети ОС в образах має перевірити сканер банку.
- **LDAP/OIDC наживо не піднімались.** VIS-SEC-044, 047, 050 і 051 перевірено за кодом; VIS-SEC-050 — ще й виконанням рядкової операції.
- **DoS-тести обмежено одним запитом на кілька секунд** (правило 3 промту). Найбільші розміри атак екстрапольовано, а не запущено.
- **Gradle:** повний транзитивний граф SDK і контрольну суму `gradle-wrapper.jar` не звірено (як і 2026-11-05).

3. Архітектура та модель загроз

Модель загроз (`audit/threat-model.md`, діаграма нижче) лишається чинною: межі довіри й потоки даних не змінились. З 7b13af1 додались дві нові поверхні:

- `GET /projects/:id/jira/types?url=…` (налаштування проєкту, #27): токен Jira йде на ще не збережену адресу (VIS-SEC-048);
- `GET /test-runs/:id/trend` (огляд, #24–#25): IDOR не знайдено.

Важливе уточнення до моделі: тепер найслабша межа — **«кілька команд в одному процесі API»** за доступністю. Обчислення над деревом екрана (VIS-SEC-043, 046) і декодування PNG (VIS-SEC-045) досі йдуть у головному потоці або без перевірки справжніх розмірів. Будь-хто з API-ключем однієї команди зупиняє Vision для всіх.



4. Регресія: знахідки аудиту 2026-11-05 і S-1…S-15

4.1. Знахідки VIS-SEC-001…042

ID	Назва	Було	Зараз	Стан	Коментар
VIS-SEC-001	Масове присвоєння в PATCH /builds/:id: ескалація editor→admin і крадіжка даних між командами	Critical	—	Закрито	Обходу не знайдено. Залишкова зміна поведінки: `{"isRunning":"false"}` (рядок) тепер 400 від ValidationPipe, а не запис — для випущених SDK (булеві значення або порожнє тіло) не впливає.
VIS-SEC-002	Приймаються відомі/слабкі секрети підпису; DEPLOY.md радить секрет робочого VRT	High	**High**	Відкладено	Відкладено рішенням власника; без змін: слабкі/відомі секрети приймаються, порада взяти секрет проду в DEPLOY.md лишилась.
VIS-SEC-003	VLM-рушій автоматично надсилає знімки банку в Gemini при прийомі (напруга з правилом 6 / ADR-0007)	High	**High**	Прийнятий ризик	Прийнятий ризик (контракт з Gemini). Лишається: (1) немає вимикача рівня інстансу на випадок відкликання погодження;
VIS-SEC-004	Обчислювальна DoS: аналіз доступності UI-дерева блокує event loop для всіх команд	High	**Medium**	Частково	(1) Решта роботи прийому з деревом лишилась у головному потоці: розбір XML (ui-tree.service.ts:276-290, parseUiTree), JSON.stringify+gzipSync дерева (293-298), декодування PNG знімка, еталона і еталона іншого пристрою (analyze → decode → hdd.service.ts:53 read…
VIS-SEC-005	Області ігнорування без меж координат зависають воркер порівняння без тайм-ауту	High	**Medium**	Частково	Кількість областей не обмежена: CreateTestRequestDto.ignoreAreas — лише @IsArray/@ValidateNested (backend/src/test-runs/dto/create-test-request.dto.ts:38-43), UpdateIgnoreAreasDto без ArrayMaxSize (update-ignore-area.dto.ts:11-14), IgnoreAreaDto лише @IsNumber…
VIS-SEC-006	Режим LDAP: будь-який користувач переприв’язує свій акаунт до чужої особи з каталогу	High	**Low**	Частково	(1) Прив'язка до акаунта досі за email, не за незмінним ідентифікатором каталогу (objectGUID/sAMAccountName): якщо банк повторно видасть адресу (новий співробітник з тим самим ім'ям), він увійде в старий акаунт звільненого (ldapusers.service.ts:179-183).
VIS-SEC-007	Вхід через LDAP з порожнім паролем не відхиляється (ризик unauthenticated bind)	High	—	Закрито	Обходу не знайдено: String.prototype.trim прибирає й Unicode-пробіли (NBSP, U+2028, BOM); символ нульової ширини (U+200B) лишає пароль непорожнім — це вже автентифікований bind, а не unauthenticated (RFC 4513 §5.1.2 — лише нульова довжина).
VIS-SEC-008	Allowlist Jira перевіряє лише hostname: проходять http, будь-який порт і редиректи	Medium	**Medium**	Не виправлено	Нова поверхня з #23–#31: GET /projects/:id/jira/types?url=… (projects.controller.ts:63-73 → jira-access.ts:52-65) бере НЕЗБЕРЕЖЕНУ адресу з query і шле на неї особистий токен після тієї ж перевірки лише hostname: проходять http://дозволений-хост:будь-який-порт…
VIS-SEC-009	Немає відкликання сесій: logout, зміна пароля, блокування й зміна ролі не гасять JWT і сокет	Medium	**Medium**	Не виправлено	Частково пом'якшено лише для доступу до проєкту: leaveProject (events.gateway.ts:115, projects.controller.ts:152) виводить сокети з кімнати при знятті доступу.

VIS-SEC-010	Дефолтний адмін із паролем 123456: нижче політики, друкується в лог, без примусової зміни	Medium	**Medium**	Не виправлено	Пароль 123456 нижчий за REQ-045-04 (12 символів) і потрапляє відкритим текстом у `docker logs` контейнера міграції. Примусової зміни немає.
VIS-SEC-011	Немає CSP і HSTS; поставка публікує HTTP на всіх інтерфейсах без TLS-термінації	Medium	**Medium**	Не виправлено	Без CSP будь-який XSS забирає JWT з localStorage (S-13); без TLS — JWT, паролі й API-ключі в мережі банку відкритим текстом. Диф 7b13af1..HEAD цих файлів не змінював.
VIS-SEC-012	Глобальний ValidationPipe без whitelist/ forbidNonWhitelisted/ transform (системний корінь mass assignment)	Medium	**—**	Закрито	Свідомі межі, не обхід: (1) лише верхній рівень — вкладені DTO (@ValidateNested: UpdateIgnoreAreasDto.ignoreAreas, CreateTestRequestDto.ignoreAreas) зберігають невідомі ключі;
VIS-SEC-013	Бекап у Google Drive не шифрується і містить тексти екранів, email і токени публічних звітів	Medium	**Medium**	Не виправлено	Без змін; ADR-0012 не переглянуто для контуру.
VIS-SEC-014	Немає меж розміру файла й розмірів PNG: multer без limits, повне декодування в головному потоці	Medium	**Medium**	Частково	Перевіряється лише перший IHDR; декодер (pngjs PNG.sync.read) потім бачить інше: повторний IHDR, черезрядкове (interlace) зображення з бомбою в IDAT, 16-бітна глибина, і канва max(w)×max(h) при allowDiffDimensions — див. NEW-C-3.
VIS-SEC-015	ReDoS: регулярний вираз із text-селектора правила елемента виконується над текстами дерева в головному потоці	Medium	**Medium**	Не виправлено	Виправлення не було. (a+)+$ вміщається в 300 символів; текст до 2000 символів дає катастрофічний бектрекінг на кожному порівнянні й відкритті structure. Суміжне — алгоритмічна складність XPath-селекторів (NEW-C-5).
VIS-SEC-016	AI_PROVIDER=none не вимикає зовнішній AI: особисті ключі Gemini і ключ проєкту діють далі	Medium	**Medium**	Не виправлено	Єдиного вимикача зовнішнього AI немає.
VIS-SEC-017	Адмін задає пароль будь-якому акаунту (зокрема SSO/LDAP) і діє його токеном Jira	Medium	**Medium**	Не виправлено	У режимі SSO заданий адміном пароль одразу діє через локальну форму (users.service.ts:127); у режимі LDAP — щойно людини немає в каталозі (NEW-A-1). Аудит-логу адмінських дій немає.
VIS-SEC-018	Login CSRF / фіксація сесії: /login#token=<будь-який JWT> входить без state	Medium	**Medium**	Не виправлено	Login CSRF/фіксація сесії лишається: посилання https://vision/login#token=<JWT нападника> мовчки входить жертвою в акаунт нападника (жертва далі завантажує туди дані/рішення, вводить свій токен Jira у профіль — PUT /users/jira пише в акаунт нападника).
VIS-SEC-019	Dev-compose у дереві: Postgres на 0.0.0.0 з postgres/postgres, ім'я контейнера збігається з продом, моки LDAP/OIDC	Medium	**Medium**	Не виправлено	Без змін.
VIS-SEC-020	SSO повертає роль editor тим, кого адмін понизив до guest; generic 'admin' у типовій групі	Medium	**Medium**	Не виправлено	Понижений адміном до guest при наступному SSO-вході знову editor/admin; група IdP з назвою «admin» (будь-якої системи) дає адміна Vision.
VIS-SEC-021	Запобіжник агентів guard.py легко обходиться — правила 1–4 тримаються лише на дисципліні	Medium	**Medium**	Не виправлено	Без змін; хук — зручність, не контроль.

VIS-SEC-022	Підписане посилання на знімок безстрокове й не прив'язане до користувача	Low	**Low**	Не виправлено	Посилання безстрокове й не відкликається, доки існує файл; FILE_CACHE 'private, max-age=31536000, immutable' (static.controller.ts:16) тримає знімок у кеші браузера рік. Для звіту підпис прив'язаний до токена звіту (signReportFile:31-32) — це гаразд.
VIS-SEC-023	Підробка записів у лозі через назву перевірки	Low	**Low**	Не виправлено	Без змін: перенос рядка в назві перевірки дає підроблений рядок логу.
VIS-SEC-024	Шифрування секретів без AAD, KDF = один SHA-256; приймається вже зашифроване значення	Low	**Low**	Не виправлено	Без змін.
VIS-SEC-025	Токени публічних звітів і токени Google Drive зберігаються відкрито	Low	**Low**	Не виправлено	Без змін.
VIS-SEC-026	AI-ендпоінти на вимогу доступні ролі guest (платні виклики й дані проєкту)	Low	**Low**	Не виправлено	Guest проєкту запускає LLM-виклики ключем проєкту/інстансу (правило 6: на вимогу, але не обмежено роллю). Ізоляція проєктів при цьому працює (@ProjectFrom на buildId/testRunId).
VIS-SEC-027	У логи потрапляють відповіді моделі (тексти екранів) і URL із підписами/токенами звітів	Low	**Low**	Не виправлено	Без змін.
VIS-SEC-028	CLI пересилає заголовок apiKey при редиректі на інший хост/схему	Low	**Low**	Не виправлено	Без змін.
VIS-SEC-029	test:pw:install вимикає перевірку TLS під час завантаження браузера	Low	**Low**	Не виправлено	Без змін.
VIS-SEC-030	CI: actions за тегами, не за SHA; не обмежено permissions GITHUB_TOKEN	Low	**Low**	Не виправлено	Без змін.
VIS-SEC-031	Gradle wrapper 6.1 без distributionSha256Sum і без перевіреної контрольної суми jar	Low	**Low**	Не виправлено	Без змін.
VIS-SEC-032	Prompt injection: тексти екрана й назви перевірок у промптах; вердикт VLM сам ставить статус	Low	**Low**	Не виправлено	Вердикт моделі не лише ставить ok, а й автоматично затверджує знімок у гілці (NEW-D-1).
VIS-SEC-033	Енумерація користувачів за часом відповіді входу	Low	**Low**	Не виправлено	Час відповіді й далі видає наявність акаунта. SignInLimit маскує лише 429, не час.
VIS-SEC-034	Ліміт входу лише за email: password spraying і блокування чужих акаунтів	Low	**Low**	Не виправлено	Password spraying (один пароль, напр. 123456 з VRT, по всіх скопійованих email) не обмежений; анонім блокує будь-який відомий email на 15 хв. Сам лічильник — нова DoS-поверхня (NEW-A-2).
VIS-SEC-035	Звільнені з каталогу/IdP зберігають доступ через API-ключ і локальний вхід	Low	**Medium**	Не виправлено	Погіршено виправленням 41b15e0: раніше для відсутньої в каталозі людини findUserInLdap кидав 400 і вхід не проходив; тепер isInDirectory=false переводить її на локальний пароль — зокрема скопійований з VRT або встановлений самою людиною живим токеном.

VIS-SEC-036	PUT /users записує email без нормалізації й перевірки дублікатів в іншому регістрі	Low	**Low**	Частково	У режимі БД можна мати «Ann@bank.ua» поруч з «ann@bank.ua»; людина з великою літерою в email не входить (а спроби рахуються лімітом за lowercase-ключем). Унікального індексу без регістру немає.
VIS-SEC-037	Скрипти запускають образи за тегом з обліковими даними проду і --network host	Low	**Low**	Не виправлено	Без змін (compose-образи postgres/redis уже за digest).
VIS-SEC-038	vision-safety.sh: .env.demo виконується як shell і може перевизначити закріплені VISION_*	Low	**Low**	Не виправлено	Без змін.
VIS-SEC-039	Образи не містять LICENSE і NOTICE (Apache-2.0 §4(a), (d))	Info	**Info**	Не виправлено	Без змін; додатково шрифти @fontsource (OFL-1.1) потрапляють у build фронтенду без тексту ліцензії (frontend/src/index.tsx:14-22).
VIS-SEC-040	JWT без явного алгоритму, issuer і audience	Info	**Info**	Не виправлено	Додатково: JWT_LIFE_TIME читається без типового значення (auth.module.ts:17) і не валідується при старті; поведінка при відсутній змінній (токен без exp або помилка підпису) — див. needs_verification. Ідентичність у токені — змінний email без sub (NEW-A-3).
VIS-SEC-041	Editor може змінювати налаштування й видаляти будь-який доступний проєкт (зокрема ключ Gemini)	Info	**Info**	Не виправлено	Editor з доступом до проєкту змінює його налаштування (Jira URL у межах VISION_JIRA_HOSTS, ключ Gemini, правила, allowedDevices) і видаляє проєкт з усіма даними. Узгоджено з глобальною моделлю ролей 042; потребує рішення/специфікації про роль на рівні проєкту.
VIS-SEC-042	Тестові фікстури містять фотографію реальних людей (успадковано від upstream)	Low	**Low**	Не виправлено	Без змін.

4.2. S-1…S-15 (аудит 2026-11-04)

Стан за S-1…S-15 з розділу 4 звіту 2026-11-05 не змінився, крім таких пунктів:

- **S-7.** `ValidationPipe` без whitelist (VIS-SEC-012) закрито: підтверджено на стенді. CSP і HSTS досі немає (VIS-SEC-011).
- **S-1.** Захист від XXE, billion laughs і gzip-бомби в дереві працює. Але нові DoS на тих самих деревах — VIS-SEC-043 і 046.
- **S-12, S-13, S-14 (свідомі рішення), переоцінка для банку:**
  - **S-12:** OIDC-частина прийнятна (Low). У LDAP `LDAP_TLS_NO_VERIFY` вимикає перевірку на весь клієнт, а `ldap://` приймається мовчки (VIS-SEC-051).
  - **S-13:** ризик вищий за оцінку 2026-11-04. CSP немає, відкликання токенів немає.
  - **S-14:** cost 10 прийнятний для нових паролів. Ризик — короткі успадковані хеші з VRT, які з VIS-SEC-044 діють і повз SSO.

Специфікації.

- **043 і 044:** усі REQ мають тести з `design.md`, код їм відповідає.
- **045:** REQ-045-01…08 реалізовані й мають тести. Але припущення дизайну REQ-045-01 («у людей SSO немає пароля, якого хтось знає») хибне для акаунтів, скопійованих з VRT (VIS-SEC-044).

5. Зведена таблиця знахідок

Відкриті знахідки за severity. Колонка «Стан»: нова, не виправлено, частково, прийнятий ризик, відкладено. Закриті — наприкінці таблиці.

ID	Назва	Severity	Стан	Компонент	Зус.
VIS-SEC-002	Приймаються відомі/слабкі секрети підпису; DEPLOY.md радить секрет робочого VRT	High	Відкладено	backend	S
VIS-SEC-003	VLM-рушій автоматично надсилає знімки банку в Gemini при прийомі (напруга з правилом 6 / ADR-0007)	High	Прийнятий ризик	backend	M
VIS-SEC-043	Квадратична побудова елементів для вибору (structure) блокує event loop на хвилини одним широким UI-деревом	High	Нова	backend	M
VIS-SEC-044	Локальний пароль діє для людей каталогу й SSO: паролі з робочого VRT обходять SSO, а звільнені з каталогу входять і самі ставлять собі пароль	High	Нова	backend	M
VIS-SEC-004	Обчислювальна DoS: аналіз доступності UI-дерева блокує event loop для всіх команд	Medium	Частково	backend	M
VIS-SEC-005	Області ігнорування без меж координат зависають воркер порівняння без тайм-ауту	Medium	Частково	backend	S
VIS-SEC-008	Allowlist Jira перевіряє лише hostname: проходять http, будь-який порт і редиректи	Medium	Не виправлено	backend	S
VIS-SEC-009	Немає відкликання сесій: logout, зміна пароля, блокування й зміна ролі не гасять JWT і сокет	Medium	Не виправлено	backend	M
VIS-SEC-010	Дефолтний адмін із паролем 123456: нижче політики, друкується в лог, без примусової зміни	Medium	Не виправлено	prisma	S
VIS-SEC-011	Немає CSP і HSTS; поставка публікує HTTP на всіх інтерфейсах без TLS-термінації	Medium	Не виправлено	infra	M
VIS-SEC-013	Бекап у Google Drive не шифрується і містить тексти екранів, email і токени публічних звітів	Medium	Не виправлено	backend	M
VIS-SEC-014	Немає меж розміру файла й розмірів PNG: multer без limits, повне декодування в головному потоці	Medium	Частково	backend	S
VIS-SEC-015	ReDoS: регулярний вираз із text-селектора правила елемента виконується над текстами дерева в головному потоці	Medium	Не виправлено	backend	M
VIS-SEC-016	AI_PROVIDER=none не вимикає зовнішній AI: особисті ключі Gemini і ключ проєкту діють далі	Medium	Не виправлено	backend	M
VIS-SEC-017	Адмін задає пароль будь-якому акаунту (зокрема SSO/LDAP) і діє його токеном Jira	Medium	Не виправлено	backend	M
VIS-SEC-018	Login CSRF / фіксація сесії: /login#token=<будь-який JWT> входить без state	Medium	Не виправлено	frontend	S
VIS-SEC-019	Dev-compose у дереві: Postgres на 0.0.0.0 з postgres/postgres, ім'я контейнера збігається з продом, моки LDAP/OIDC	Medium	Не виправлено	infra	S
VIS-SEC-020	SSO повертає роль editor тим, кого адмін понизив до guest; generic 'admin' у типовій групі	Medium	Не виправлено	backend	M
VIS-SEC-021	Запобіжник агентів guard.py легко обходиться — правила 1–4 тримаються лише на дисципліні	Medium	Не виправлено	ci	M
VIS-SEC-035	Звільнені з каталогу/IdP зберігають доступ через API-ключ і локальний вхід	Medium	Не виправлено	backend	S
VIS-SEC-045	Обхід ліміту пікселів: перевіряється лише перший IHDR, а pngjs розпаковує інше (повторний IHDR, interlace-бомба, канва різних розмірів)	Medium	Нова	backend	S

VIS-SEC-046	Розбір XML, серіалізація дерева і декодування PNG при прийомі досі в головному потоці; глибина дерева дає ×200 підсилення	Medium	Нова	backend	M
VIS-SEC-006	Режим LDAP: будь-який користувач переприв’язує свій акаунт до чужої особи з каталогу	Low	Частково	backend	M
VIS-SEC-022	Підписане посилання на знімок безстрокове й не прив'язане до користувача	Low	Не виправлено	backend	S
VIS-SEC-023	Підробка записів у лозі через назву перевірки	Low	Не виправлено	backend	S
VIS-SEC-024	Шифрування секретів без AAD, KDF = один SHA-256; приймається вже зашифроване значення	Low	Не виправлено	backend	S
VIS-SEC-025	Токени публічних звітів і токени Google Drive зберігаються відкрито	Low	Не виправлено	backend	S
VIS-SEC-026	AI-ендпоінти на вимогу доступні ролі guest (платні виклики й дані проєкту)	Low	Не виправлено	backend	S
VIS-SEC-027	У логи потрапляють відповіді моделі (тексти екранів) і URL із підписами/токенами звітів	Low	Не виправлено	backend	S
VIS-SEC-028	CLI пересилає заголовок apiKey при редиректі на інший хост/схему	Low	Не виправлено	cli	S
VIS-SEC-029	test:pw:install вимикає перевірку TLS під час завантаження браузера	Low	Не виправлено	frontend	S
VIS-SEC-030	CI: actions за тегами, не за SHA; не обмежено permissions GITHUB_TOKEN	Low	Не виправлено	ci	S
VIS-SEC-031	Gradle wrapper 6.1 без distributionSha256Sum і без перевіреної контрольної суми jar	Low	Не виправлено	sdk-java	S
VIS-SEC-032	Prompt injection: тексти екрана й назви перевірок у промптах; вердикт VLM сам ставить статус	Low	Не виправлено	backend	S
VIS-SEC-033	Енумерація користувачів за часом відповіді входу	Low	Не виправлено	backend	S
VIS-SEC-034	Ліміт входу лише за email: password spraying і блокування чужих акаунтів	Low	Не виправлено	backend	S
VIS-SEC-036	PUT /users записує email без нормалізації й перевірки дублікатів в іншому регістрі	Low	Частково	backend	S
VIS-SEC-037	Скрипти запускають образи за тегом з обліковими даними проду і --network host	Low	Не виправлено	scripts	S
VIS-SEC-038	vision-safety.sh: .env.demo виконується як shell і може перевизначити закріплені VISION_*	Low	Не виправлено	scripts	S
VIS-SEC-042	Тестові фікстури містять фотографію реальних людей (успадковано від upstream)	Low	Не виправлено	backend	S
VIS-SEC-047	JWT прив'язаний лише до змінного email: у режимі БД можна заздалегідь «накарбувати» токен для чужої майбутньої адреси	Low	Нова	backend	S
VIS-SEC-048	Новий GET /projects/:id/jira/types надсилає особистий токен Jira на адресу з query-рядка; перевірка проєкту декоративна (розширення VIS-SEC-008)	Low	Нова	backend	S
VIS-SEC-049	Лічильник спроб входу: квадратична CPU-DoS і необмежене зростання пам'яті від анонімних запитів	Low	Нова	backend	S
VIS-SEC-050	Фільтр LDAP: String.replaceAll трактує $', $` і $& з email як шаблони заміни — обхід екранування	Low	Нова	backend	S
VIS-SEC-051	LDAP: вимкнення перевірки TLS діє на весь клієнт без попередження, ldap:// без TLS приймається мовчки	Low	Нова	backend	S
VIS-SEC-052	Три падіння воркера поспіль (зокрема повтори одного завдання) назавжди переводять пул порівняння в головний потік без тайм-ауту	Low	Нова	backend	S

VIS-SEC-053	XPath-селектори правил з осями // і ancestor мають необмежену складність і виконуються в головному потоці	Low	Нова	backend	S
VIS-SEC-054	Тексти з UI-дерева і назва перевірки потрапляють у wiki-розмітку Jira без екранування	Low	Нова	backend	S
VIS-SEC-055	VLM-вердикт при прийомі запускає автоапрув і множить виклики Gemini без ліміту	Low	Нова	backend	S
VIS-SEC-039	Образи не містять LICENSE і NOTICE (Apache-2.0 §4(a),(d))	Info	Не виправлено	infra	S
VIS-SEC-040	JWT без явного алгоритму, issuer і audience	Info	Не виправлено	backend	S
VIS-SEC-041	Editor може змінювати налаштування й видаляти будь-який доступний проєкт (зокрема ключ Gemini)	Info	Не виправлено	backend	M
VIS-SEC-056	StripUnknownFieldsPipe не чистить вкладені DTO: зайві ключі областей ігнорування зберігаються в JSON	Info	Нова	backend	S
VIS-SEC-057	Застарілі облікові дані й чужа IP-адреса в документації та тестах	Info	Нова	infra	S
VIS-SEC-001	Масове присвоєння в PATCH /builds/:id: ескалація editor→admin і крадіжка даних між командами	—	Закрито	backend	S
VIS-SEC-007	Вхід через LDAP з порожнім паролем не відхиляється (ризик unauthenticated bind)	—	Закрито	backend	S
VIS-SEC-012	Глобальний ValidationPipe без whitelist/forbidNonWhitelisted/transform (системний корінь mass assignment)	—	Закрито	backend	S

6. Детальні знахідки

Спершу 15 нових знахідок (VIS-SEC-043…057) — їх аудит 2026-11-05 не містив. Далі знахідки попереднього аудиту, які змінили стан або severity. Повний опис незмінених знахідок — у `audit/report-2026-11-05.pdf`; актуальний стан кожної з них є в розділі 4 і в `audit/findings.json`.

6.1. Нові знахідки

High

High VIS-SEC-043 · Нова **Квадратична побудова елементів для вибору (structure) блокує event loop на хвилини одним широким UI-деревом**

CWE / OWASP	CWE-407 · A04:2021-Insecure Design
CVSS	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:H (6.5)`
Компонент / тип	backend · code · зусилля M · впевненість High

Місце	`backend/src/ui-tree/ui-tree.service.ts:154-186`
	`backend/src/ui-tree/ui-tree.service.ts:485`
	`backend/src/ui-tree/ui-tree.service.ts:535`
	`backend/src/ui-tree/selector-variants.ts:86-90`
	`backend/src/ui-tree/selector-variants.ts:97-106`
	`backend/src/ui-tree/selector-variants.ts:113-145`
	`backend/src/ui-tree/xpath-selector.ts:171-179`
	`backend/src/test-runs/test-runs.controller.ts:246`

Опис. GET /test-runs/:id/structure (відкриття знімка в інтерфейсі) синхронно в головному потоці викликає pickableElements для дерева і для дерева еталона. Для кожного безіменного видимого вузла selectorVariants будує шлях через stepTo, який фільтрує ВСІХ сиблінгів (siblings.filter(...).indexOf), і одразу виконує XPath через findBySelector, де крок child знову фільтрує всіх дітей батька (children[parent].filter). Для текстових вузлів те саме робить pathFromIdAbove. На дереві, де під одним вузлом з id лежать n безіменних сиблінгів, це ~2·n² операцій на дерево. Обмеження 10 МиБ (040) дозволяє сотні тисяч вузлів (`<n bounds="[0,0][1,2]"/>` ≈ 25 байт).

Доказ. Підтверджено на стенді (repro-005): дерево з 10 000 безіменних сусідніх вузлів (826 КБ XML, 110 КБ у gzip+base64) — GET /test-runs/:id/structure 6,8 с, паралельний GET /users/me чекав 5,8 с (event loop заблоковано для всіх команд). Складність квадратична: 30 000 вузлів — орієнтовно хвилина на кожне відкриття знімка. selector-variants.ts:86-90: `const siblings = childrenOf(tree)[node.parent]; const position = siblings.filter((index) => tree.nodes[index].className === node.className).indexOf(node.index) + 1;` викликається для кожного вузла з ui-tree.service.ts:162-163 (`selectorVariants(tree, node)`) і 181 (`textFields` → pathFromIdAbove). xpath-selector.ts:177: `const matching = children[parent].filter((child) => fits(tree.nodes[child], step.test));`. Офлайн-симуляція саме цих двох циклів (node 22, scratchpad sim/wide.js): 10 000 сиблінгів — 1,96 с, 30 000 — 28,9 с (≈1,7 МБ XML); 100 000 сиблінгів екстраполюється в ~5 хв; викликається двічі (tree і baselineTree). Живий запит не виконувався.

Вплив. Будь-хто з правом завантаження (API-ключ CI, editor) один раз надсилає знімок із широким деревом; кожне відкриття цього знімка будь-яким переглядачем (і відкриття знімків, для яких дерево стало еталоном) зупиняє API для всіх команд на десятки секунд–хвилини. Повторні відкриття множать ефект.

Рекомендація. Обчислювати позицію серед сиблінгів одним проходом (Map className→лічильник для кожного батька, кешувати як childrenOf); не перевіряти кожен варіант повним XPath, або кешувати результати; обмежити кількість вузлів дерева при прийомі (напр. 20–50 тис.) і кількість елементів, для яких будуються варіанти; винести structure у воркер з тайм-аутом. Тест: дерево з 50 000 сиблінгів відповідає structure за < 1 с.

Пов’язано VIS-SEC-004, NEW-C-2, NEW-C-5

High VIS-SEC-044 · Нова **Локальний пароль діє для людей каталогу й SSO: паролі з робочого VRT обходять SSO, а звільнені з каталогу входять і самі ставлять собі пароль**

CWE / OWASP	CWE-288 · A07:2021-Identification and Authentication Failures
CVSS	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N (7.4)`
Компонент / тип	backend · code · зусилля M · впевненість High

Місце	`backend/src/users/users.service.ts:123-129`
	`backend/src/users/users.service.ts:77-85`
	`backend/src/users/ldap/ldapusers.service.ts:54-67`
	`backend/src/users/db/dbusers.service.ts:58-74`
	`docs/specs/045-sign-in-and-api-hardening/design.md:5-11`
	`DEPLOY.md:124-126`
	`DEPLOY.md:174-175`

Опис. У User немає ознаки походження (локальний / каталог / SSO). Рішення «каталог чи локальний пароль» приймається на кожен вхід за поточною відповіддю каталогу, а в режимі SSO форма пароля працює для будь-якого email. Дизайн 045 стверджує: «Акаунти з каталогу й SSO мають пароль, якого ніхто не знає, тож локальний вхід їм не відкривається» — це неправда для акаунтів, скопійованих з робочого VRT разом з хешами паролів (DEPLOY.md: «Копируется вся база VRT … пользователи с паролями»), і для паролів, заданих адміном (VIS-SEC-017). Виправлення 41b15e0 (isInDirectory=false замість 400) розширило це на LDAP: людина, яку прибрали з каталогу або перенесли за межі LDAP_SEARCH_DN (типово для блокування в AD), стає «локальною».

Доказ. Перевірено за кодом: users.service.ts:123-129 — у режимі SSO форма завжди йде в localUsers.login; у режимі LDAP — щойно isInDirectory=false. Ознаки походження акаунта немає (schema.prisma, User). Специфікація 045 (REQ-045-01) пускає «локальні акаунти, створені адміном», але код не відрізняє їх від скопійованих з VRT. users.service.ts:124-127: if (this.usersService instanceof LdapUsersService && !(await this.usersService.isInDirectory(dto.email))) { return this.localUsers.login(dto); } if (this.usersService instanceof OidcUsersService) return this.localUsers.login(dto); users.service.ts:81-83 — зміна пароля: якщо людини немає в каталозі, `this.localUsers.changePassword(user, newPassword)`. ldapusers.service.ts:64 — «не знайдено» і «бракує атрибута» (400) -> null -> «не в каталозі». DbUsersService.login (dbusers.service.ts:58-70) перевіряє збережений хеш, яким би шляхом він не з'явився (скопійований з VRT, від адміна).

Вплив. (1) SSO-режим: кожен користувач VRT входить старим паролем VRT повз SSO/MFA банку; паролі VRT не проходили політику 12 символів (spec 045: короткий наявний пароль працює), тож можливий spraying (VIS-SEC-034) по всіх скопійованих email. (2) LDAP-режим: звільнений, якого вивели з області пошуку, входить старим паролем VRT; а з ще живим JWT (до 1 доби, VIS-SEC-009) може викликати PUT /users/password і поставити собі постійний локальний пароль — безстроковий доступ до знімків банківських застосунків. (3) Те саме для акаунтів VRT з неіснуючими в каталозі адресами (тестові, сервісні, дефолтний адмін VRT — див. needs_verification).

Рекомендація. Потребує специфікації. Додавальна nullable-колонка походження акаунта (authSource: local|ldap|oidc, ADR-0003); локальний вхід і зміну пароля дозволяти лише для local. При переході з VRT у режимі LDAP/SSO — анулювати скопійовані хеші людей каталогу/IdP (або позначити їх як ldap/oidc). Відсутність людини в каталозі для акаунта з authSource=ldap — відмова у вході й блокування, а не локальний пароль. Заборонити setPassword для не-local (VIS-SEC-017).

Пов’язано VIS-SEC-035, VIS-SEC-017, VIS-SEC-009, VIS-SEC-034, VIS-SEC-006

Medium

Medium **VIS-SEC-045** · Нова · VIS-SEC-014 **Обхід ліміту пікселів: перевіряється лише перший IHDR, а pngjs розпаковує інше (повторний IHDR, interlace-бомба, канва різних розмірів)**

CWE / OWASP	CWE-409 · A04:2021-Insecure Design
CVSS	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:H (6.5)`
Компонент / тип	backend · code · зусилля S · впевненість High

Місце	`backend/src/shared/screenshot-limits.ts:31-35`
	`backend/src/shared/screenshot-limits.ts:63-75`
	`backend/src/static/hdd/hdd.service.ts:53`
	`backend/src/compare/libs/pixelmatch/pixelmatch.core.ts:175-199`
	`backend/src/compare/utils/index.ts:5-12`

Опис. readPngSize читає ширину/висоту з байтів 16–23 і перевіряє лише, що на зсуві 12 є 'IHDR'; довжину чанка, CRC, наступні чанки, interlace і глибину не дивиться. readPng після цього викликає PNG.sync.read на тих самих байтах. За кодом pngjs 6/7 (parser.js/parser-sync.js — не перевірено локально, node_modules немає): (а) другий IHDR не відкидається, а метадані перезаписуються останнім, тож заголовок 1×1, а розпаковується 20000×20000; (б) для interlace=1 IDAT розпаковується zlib.inflateSync без maxLength — кілька МБ нулів у IDAT дають гігабайти; (в) при 16-бітній глибині проміжні буфери 8 байт/піксель, а не 4, як припускає коментар. Окремо: при allowDiffDimensions (налаштування проєкту) pixelmatch.core.ts будує канву max(w)×max(h) — знімок 20000×2000 і еталон 2000×20000 кожен у межах 40 Мпкс, але scaleImageToSize і diff дають три буфери по 1,6 ГБ.

Доказ. Підтверджено наживо (repro-005): PNG 560 КБ з першим IHDR 10×10 і другим 12000×12000 (144 Мпкс, у 3,6 раза понад ліміт) прийнято (201); при порівнянні з еталоном RSS процесу API зріс до 2,4 ГБ. Офлайн: 249 КБ → 8000×8000, 256 МБ RGBA за 1,2 с (readPng пропускає). Розміри 20000×20000 не пробувались (обмеження DoS-тестів), але за пропорцією це кілька ГБ — падіння процесу. screenshot-limits.ts:34: `if (!bytes.subarray(0, 8).equals(PNG_SIGNATURE) || bytes.toString('latin1', 12, 16) !== 'IHDR') return null; return { width: bytes.readUInt32BE(16), height: bytes.readUInt32BE(20) };` і :74 `return PNG.sync.read(buffer);`. pixelmatch.core.ts:189-190: `const maxWidth = Math.max(baseline.width, image.width); const maxHeight = Math.max(baseline.height, image.height);` → index.ts:7 `new PNG({ width, height, fill: true })`.

Вплив. Буфери поза V8 heap (ArrayBuffer) не обмежуються ліміт heap воркера: виділення в гігабайти у воркері або в головному потоці (decode при analyze) виснажує пам'ять контейнера і OOM-killer вбиває весь API (усі команди). Виправлення VIS-SEC-014 обходиться файлом у кілька кілобайт–мегабайт.

Рекомендація. Перед декодуванням пройти всі чанки: рівно один IHDR довжини 13 першим, перевірити CRC, interlace=0 (або обмежити розпаковку), bit depth ≤ 8 (або рахувати байти/піксель), відкидати після IEND. Розпаковувати IDAT з maxOutputLength = очікуваний розмір. Обмежити max(w)×max(h) канви при allowDiffDimensions тим самим лімітом. Додати тести: PNG з двома IHDR, interlace-бомба, 16-біт, різні орієнтації.

Пов’язано	VIS-SEC-014, NEW-C-2

Medium **VIS-SEC-046** · Нова · VIS-SEC-004 **Розбір XML, серіалізація дерева і декодування PNG при прийомі досі в головному потоці; глибина дерева дає ×200 підсилення**

CWE / OWASP	CWE-400 · A04:2021-Insecure Design
CVSS	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:H (6.5)`
Компонент / тип	backend · code · зусилля M · впевненість Medium

Місце	`backend/src/ui-tree/ui-tree.service.ts:276-298`
	`backend/src/ui-tree/ui-tree.parser.ts:20-27`
	`backend/src/ui-tree/ui-tree.parser.ts:166-190`
	`backend/src/ui-tree/ui-tree.parser.ts:220-228`
	`backend/src/ui-tree/ui-tree.service.ts:305-320`
	`backend/src/ui-tree/ui-tree.service.ts:336-349`
	`backend/src/static/hdd/hdd.service.ts:49-57`
	`backend/src/ui-tree/ui-tree.intake.ts:20-38`

Опис. Після 88669e6 у воркер пішли лише правила доступності. При кожному прийомі з деревом у головному потоці синхронно: fast-xml-parser.parse до 10 МиБ XML, рекурсивний walk, обчислення ключів `path:${node.path}`, JSON.stringify і gzipSync усього дерева (store), а в analyze — декодування до трьох PNG до 40 Мпкс (знімок, еталон, еталон іншого пристрою) через StaticService.getImage → readPng → PNG.sync.read, diffTrees і componentChanges. Ключ безіменного вузла містить повний шлях від кореня, тож розмір збереженого JSON росте як глибина², а ліміт 10 МиБ рахує вхід, а не результат. Захист від XXE/billion laughs працює: DOCTYPE/ENTITY відкидається (ui-tree.intake.ts:35), gunzip обмежено maxOutputLength (24), fast-xml-parser не тягне зовнішніх сутностей.

Доказ. ui-tree.parser.ts:186: `path = ${parentPath}/${shortClass(node.className)}[${siblingIndex}];` і :222 `: path:${node.path}`;` `ui-tree.service.ts:293-298` gzipSync(JSON.stringify(tree))`. Офлайн-симуляція того самого walk/ключів/stringify/gzip (scratchpad sim/depth2.js, node 22): 12 ланцюжків глибиною 3000 (≈2,2 МБ XML, gzip ще менше) → walk+ключі 2,0 с, JSON 434 МБ за 2,6 с, gzip 1,8 с: разом 6,4 с блокування event loop і ~0,7 ГБ пам'яті на один запит без урахування самого fast-xml-parser. Глибше ≈3500 рекурсія walk падає з RangeError (перехоплюється в store:296 — дерево відкидається). Далі збережене дерево читає load() (JSON.parse 434 МБ у головному потоці, кеш до 300 дерев — ui-tree.service.ts:36,305-320) і копіює в воркер для a11y.

Вплив. Кілька паралельних завантажень на хвилину від одного CI-ключа тримають API недоступним для всіх команд; кеш дерев на 300 записів із деревами по сотні МБ веде до вичерпання heap і падіння процесу.

Рекомендація. Винести store (розбір, серіалізацію) і analyze повністю у воркер-пул із тайм-аутом; обмежити глибину (напр. 200) і кількість вузлів дерева при розборі; не тримати повний шлях у ключі (хеш шляху або індекс), обмежити розмір серіалізованого дерева; кеш дерев обмежувати за байтами. Декодування PNG для a11y робити у воркері, передавати буфер PNG, а не RGBA. Потребує специфікації (зміна поведінки прийому).

Пов’язано	VIS-SEC-004, VIS-SEC-014, NEW-C-1

Low

Low VIS-SEC-047 · Нова **JWT прив'язаний лише до змінного email: у режимі БД можна заздалегідь «накарбувати» токен для чужої майбутньої адреси**

CWE / OWASP	CWE-287 · A07:2021-Identification and Authentication Failures
CVSS	`CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:H/I:H/A:N (6.8)`
Компонент / тип	backend · code · зусилля S · впевненість Medium

Місце	`backend/src/auth/auth.service.ts:59-66`
	`backend/src/auth/jwt.strategy.ts:94-104`
	`backend/src/users/db/dbusers.service.ts:35-46`
	`backend/src/shared/events/events.gateway.ts:68-73`

Опис. Токен містить лише { email }, без sub (id), jti чи версії; JwtStrategy шукає користувача за email із токена. У режимі локальних акаунтів PUT /users (DbUsersService.update) дозволяє будь-якому користувачу поставити собі довільний ще не зайнятий email і отримати у відповіді токен на нього, а потім повернути свій email. Токен лишається дійсним до exp і автентифікує того, хто ПІЗНІШЕ отримає цю адресу: акаунт, який адмін створить новому співробітнику (передбачувана firstname.lastname@bank), відновлений після видалення акаунт, або акаунт, що з'явиться при першому вході через LDAP/SSO після зміни режиму.

Доказ. Знижено з Medium: потрібен автентифікований користувач режиму БД, знання майбутньої адреси й створення акаунта з нею в межах JWT_LIFE_TIME. auth.service.ts:60-62 `const payload: JwtPayload = { email: user.email };` jwt.strategy.ts:95-97 `findUnique({ where: { email: payload.email } })` dbusers.service.ts:36-45 `update({ where: { id }, data: { email: userDto.email, … } }); const token = this.authService.signToken(user);`

Вплив. Захоплення нового акаунта (включно з адміністраторським, якщо адмін створює нового адміна) інсайдером із будь-якою роллю протягом JWT_LIFE_TIME (типово 1d, у dev 7d). Те саме — старі токени видаленого акаунта діють для нового акаунта з тією ж адресою.

Рекомендація. Класти в JWT sub=user.id (і версію токена з VIS-SEC-009), шукати користувача за id і звіряти email; у режимі БД нормалізувати email і, за потреби, заборонити зміну email без адміна. Зворотна сумісність: старі токени (лише email) приймати до їхнього exp або примусити повторний вхід.

Пов’язано VIS-SEC-009, VIS-SEC-036, VIS-SEC-040, VIS-SEC-006

Low **VIS-SEC-048** · Нова **Новий GET /projects/:id/jira/types надсилає особистий токен Jira на адресу з query-рядка; перевірка проєкту декоративна (розширення VIS-SEC-008)**

CWE / OWASP	CWE-319 · A01:2021-Broken Access Control
CVSS	`CVSS:3.1/AV:A/AC:H/PR:L/UI:R/S:U/C:L/I:N/A:N (2.6)`
Компонент / тип	backend · code · зусилля S · впевненість High

Місце	`backend/src/projects/projects.controller.ts:63-73`
	`backend/src/projects/projects.service.ts:144-154`
	`backend/src/a11y/jira-access.ts:52-65`
	`backend/src/a11y/jira-hosts.ts:15-22`

Опис. Ендпоінт з #27 (специфікація 049) бере `url` із query (не збережений Jira URL проєкту) і відкриває JiraClient з особистим токеном того, хто дивиться сторінку. Єдина перевірка — jiraHostProblem, яка звіряє лише hostname (VIS-SEC-008): `http://`, будь-який порт і будь-який шлях на дозволеному хості проходять. Параметр `id` у сервіс не передається: він потрібен лише інтерцептору, а неіснуючий UUID інтерцептор пропускає («рядок не знайдено — лишається маршруту», project-access.interceptor.ts:46), тож editor без жодного проєкту теж викликає маршрут.

Доказ. Та сама вада, що VIS-SEC-008 (перевірка лише hostname), на новому маршруті з #27; відповідь Jira людині не повертається. projects.controller.ts:71-72: `jiraTypes(@Param('id', new ParseUUIDPipe()) id: string, @Query('url') url: string, @CurrentUser() user: User) { return this.projectsService.jiraTypes(url, user.id); }`; jira-access.ts:59-64: `const url = jiraUrl?.trim(); ... if (jiraHostProblem(url, allowedJiraHosts(...))) return { problem: 'hostNotAllowed' }; ... const token = openSecret(user?.jiraToken, env); ... return new JiraClient({ url, token, email: user.jiraEmail });`; jira-hosts.ts:19: `return hosts.includes(new URL(url).hostname.toLowerCase());`. Раніше токен ішов лише на збережений jiraUrl проєкту; тепер — на значення поля форми до збереження (frontend надсилає те, що введено).

Вплив. Чужий токен цим не отримати (тільки власний, JWT лише в заголовку Bearer — CSRF немає). Ризик: людина, що вводить `http://jira.bank/...` або адресу з нестандартним портом, ще до збереження відправляє свій токен Jira (Basic email:token або Bearer) відкритим текстом через внутрішню мережу банку; також маршрут дає перебір портів/шляхів дозволеного хоста зі статусом ok/failed. Масив `?url=a&url=b` дає 500 (TypeError на trim), не більше.

Рекомендація. Під час виправлення VIS-SEC-008 перевіряти схему https і порт (і `redirect: 'manual'`) в isAllowedJiraUrl — це закриє обидва шляхи. Валідувати `url` як рядок (@IsUrl у DTO запиту або ручна перевірка typeof). Або передавати id у сервіс і вимагати наявний проєкт, або позначити маршрут @NoProjectData, щоб не створювати хибного враження перевірки.

Пов’язано VIS-SEC-008, спека 049 REQ-049-03/09/13

Low VIS-SEC-049 · Нова **Лічильник спроб входу: квадратична CPU-DoS і необмежене зростання пам'яті від анонімних запитів**

CWE / OWASP	CWE-407 · A04:2021-Insecure Design
CVSS	`CVSS:3.1/AV:A/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H (6.5)`
Компонент / тип	backend · code · зусилля S · впевненість High
Місце	`backend/src/users/sign-in-limit.ts:291-313` `backend/src/users/users.service.ts:100-117`

Опис. Кожна невдала спроба викликає forgetOld(), який при failures.size >= 10000 перебирає ВСЮ мапу і видаляє лише записи старші за 15 хв. Анонім, надсилаючи невдалі входи з різними вигаданими email (для невідомого email у режимі БД/SSO немає bcrypt — запит дешевий), тримає мапу заповненою свіжими записами: кожен наступний запит коштує O(n) у головному потоці, сумарно O(n²). Мапа closedUntil не чиститься взагалі (запис видаляється лише при перевірці того самого email) — кожні 5 невдач по новому email додають запис назавжди до перезапуску.

Доказ. Знижено з Medium за заміром: після 10 000 записів кожна невдала спроба коштує 0,2–0,5 мс (40 000 записів), тобто для помітного блокування потрібні сотні тисяч запитів за 15 хв; пам'ять closedUntil не чиститься. sign-in-limit.ts:309-313: private forgetOld(): void { if (this.failures.size < 10000) return; const since = this.now() - SIGN_IN_WINDOW_MS; for (const [key, times] of this.failures) if (!times.some((at) => at > since)) this.failures.delete(key); } sign-in-limit.ts:295-297 — closedUntil.set(...) без жодного очищення, крім closed() для того ж ключа.

Вплив. Один процес API (spec 045: «Vision працює одним процесом API») обслуговує і UI, і прийом знімків із CI всіх команд. Сотні тисяч свіжих записів роблять кожну невдалу спробу мілісекундами синхронного CPU — event loop насичується, прийом знімків і UI зависають для всіх. У режимі LDAP кожен такий запит ще й робить bind+search у корпоративний каталог (навантаження на AD банку).

Рекомендація. Обмежити розмір обох мап (LRU/кільцевий буфер з жорсткою межею), чистити closedUntil разом із failures, прибирати застарілі записи амортизовано (за таймером раз на хвилину, а не на кожен запит). Додатково — грубий ліміт частоти /users/login на з'єднання/IP за коректним trust proxy (поза 045 — потребує рішення).

Пов’язано VIS-SEC-034, VIS-SEC-004

Low VIS-SEC-050 · Нова **Фільтр LDAP: String.replaceAll трактує $', $` і $& з email як шаблони заміни — обхід екранування**

CWE / OWASP	CWE-90 · A03:2021-Injection
CVSS	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N (3.7)`
Компонент / тип	backend · code · зусилля S · впевненість High
Місце	`backend/src/users/ldap/ldapusers.service.ts:71` `backend/src/users/ldap/ldapusers.service.ts:83`

Опис. Email екранується за RFC 4515 (* ( ) \ NUL), але далі підставляється рядком-заміною: `usersSearchFilter.replaceAll('{{email}}', email)`. У рядку заміни JS інтерпретує `$'` (текст після збігу), `` $ `(текст до збігу), $&, $$. Ці символи дозволені @IsEmail у локальній частині (validator.js допускає $ ' `` у local part). Так у фільтр потрапляють неекрановані дужки з самого шаблону, напр. для `(|(sAMAccountName={{email}})(mail={{email}}))` email `alice$'@x.yz` дає `(|(sAMAccountName=alice)(mail={{email}}))@x.yz)…`.

Доказ. Перевірено: "(mail={{email}})".replaceAll("{{email}}", "a$'b") → "(mail=a)b)". ldapusers.service.ts:83 `filter: this.ldapConfig.usersSearchFilter.replaceAll('{{email}}', email),`; escapeLdap (205-230) не чіпає `$`.

Вплив. Пошук за значенням, яке не є email (напр. sAMAccountName=alice), і структурні зміни фільтра; пароль однаково перевіряє bind, тож прямого обходу автентифікації немає. Якщо парсер ldapts приймає хвіст після збалансованого фільтра, різні «домени» дають різні ключі SignInLimit для однієї людини — необмежений підбір пароля людини каталогу (див. needs_verification). Інакше — 500 замість 400 (такі спроби не рахуються лімітом).

Рекомендація. Підставляти функцією: `replaceAll('{{email}}', () => email)`; додатково відхиляти email з `$` до пошуку; тест на `$'`/`` $ ``.

Пов’язано	VIS-SEC-034

Low VIS-SEC-051 · Нова · S-12 **LDAP: вимкнення перевірки TLS діє на весь клієнт без попередження, ldap:// без TLS приймається мовчки**

CWE / OWASP	CWE-295 · A02:2021-Cryptographic Failures
CVSS	`CVSS:3.1/AV:A/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N (4.8)`
Компонент / тип	backend · config · зусилля S · впевненість High
Місце	`backend/src/users/ldap/ldapusers.service.ts:44-51` `docker-compose.demo.yml:87,95`

Опис. На відміну від OIDC (обмежено origin issuer + warn), LDAP_TLS_NO_VERIFY=true ставить rejectUnauthorized:false для всього клієнта і нічого не пише в лог; LDAP_URL зі схемою ldap:// (без StartTLS) приймається без попередження. Через цей клієнт ідуть сервісний bind (LDAP_BIND_PASSWORD) і bind кожної людини з її паролем.

Доказ. ldapusers.service.ts:44 `tlsNoVerify: … 'LDAP_TLS_NO_VERIFY' … === 'true'`, :46-51 `new LdapClient({ url, tlsOptions: { rejectUnauthorized: !this.ldapConfig.tlsNoVerify } })` — без перевірки схеми й без logger.warn.

Вплив. Помилка конфігурації в контурі банку непомітно віддає доменні паролі співробітників і пароль сервісного акаунта каталогу на перехоплення в мережі.

Рекомендація. Попередження в лог при NO_VERIFY і ldap://; у прод-профілі — відмова стартувати без ldaps:// або StartTLS; описати в DEPLOY.md.

Пов’язано	S-12, VIS-SEC-011

Low VIS-SEC-052 · Нова · VIS-SEC-005 Три падіння воркера поспіль (зокрема повтори одного завдання) назавжди переводять пул порівняння в головний потік без тайм-ауту

CWE / OWASP	CWE-636 · A04:2021-Insecure Design
CVSS	`CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:N/I:N/A:H (5.3)`
Компонент / тип	backend · code · зусилля S · впевненість Low

Місце	`backend/src/compare/diff-worker-pool.ts:108-157`
	`backend/src/compare/diff-worker-pool.ts:159-170`
	`backend/src/compare/diff-worker-pool.ts:62-64`

Опис. Падіння воркера ('error' або exit≠0) повертає його завдання в чергу (до MAX_JOB_ATTEMPTS=2 повторів) і збільшує spawnFailures, яке скидається лише успішним повідомленням. Отже, одне вхідне завдання, що детерміновано валить воркер, сам по собі дає 3 падіння поспіль → inline=true назавжди, черга виконується синхронно в головному потоці (drainQueueInline), і всі подальші порівняння й a11y-завдання ідуть у головному потоці без тайм-ауту — VIS-SEC-004/005 повертаються до перезапуску процесу. Логіка «скрипт воркера не вантажиться» не відрізняється від «завдання вбиває воркер».

Доказ. diff-worker-pool.ts:131-133 `if (!this.destroyed && job.attempts < MAX_JOB_ATTEMPTS) { job.attempts += 1; this.queue.unshift(job);` і 145-151 `this.spawnFailures += 1; if (this.spawnFailures >= MAX_SPAWN_FAILURES) { this.inline = true; ... this.drainQueueInline();`; скидання лише в message (101).

Вплив. Якщо знайдеться вхід, що валить воркер (ERR_WORKER_OUT_OF_MEMORY на великому дереві/зображенні, нативна помилка), один запит вимикає захист від DoS для всього інстансу.

Рекомендація. Рахувати spawn failures лише для воркерів, що впали до першого завдання (або під час старту); завдання, що впало двічі, відхиляти без зміни режиму пулу; не переходити на головний потік автоматично (краще відмовляти з помилкою), або повертатись до воркерів за часом; задати resourceLimits воркерам.

Пов’язано	VIS-SEC-004, VIS-SEC-005

Low **VIS-SEC-053** · Нова **XPath-селектори правил з осями // і ancestor мають необмежену складність і виконуються в головному потоці**

CWE / OWASP	CWE-407 · A04:2021-Insecure Design
CVSS	`CVSS:3.1/AV:N/AC:L/PR:H/UI:N/S:U/C:N/I:N/A:H (4.9)`
Компонент / тип	backend · code · зусилля S · впевненість Medium

Місце	`backend/src/ui-tree/xpath-selector.ts:160-195`
	`backend/src/ui-tree/comparison-inputs.ts:152-159`
	`backend/src/ui-tree/comparison-inputs.ts:206-227`
	`backend/src/test-runs/dto/element-rule.dto.ts:13`

Опис. evaluateXpath для кожного кроку descendant для кожного вузла контексту будує descendantsOrSelf (O(розмір піддерева)), для ancestor — проходить усіх предків. Селектор до 300 символів дає десятки кроків `//*`; на глибокому/широкому дереві (до 10 МиБ) кожен крок ~n·глибина, а правило проєкту застосовується до кожного порівняння кожного знімка (resolveRules) і до structure. ADR-0021 обмежує граматику (без функцій, or, інших осей), але не вартість.

Доказ. xpath-selector.ts:165-168 `const descendantsOrSelf = (index) => { const all = [index]; for (let i = 0; i < all.length; i++) all.push(...children[all[i]]); return all; };` і 174-179 цикл по context × parents; кількість кроків обмежена лише @MaxLength(300).

Вплив. Editor проєкту може зробити кожне порівняння свого (і спільного, якщо проєкт спільний) проєкту повільним, блокуючи event loop для всіх команд; потрібна роль editor.

Рекомендація. Обмежити кількість кроків (напр. ≤ 8) і кількість descendant-кроків (≤ 2); рахувати descendant один раз на крок (позначка відвіданих), обмежити розмір контексту; виконувати резолв правил у воркері.

Пов’язано VIS-SEC-015, NEW-C-1

Low **VIS-SEC-054** · Нова **Тексти з UI-дерева і назва перевірки потрапляють у wiki-розмітку Jira без екранування**

CWE / OWASP	CWE-74 · A03:2021-Injection
CVSS	`CVSS:3.1/AV:N/AC:H/PR:L/UI:R/S:C/C:L/I:L/A:N (3.4)`
Компонент / тип	backend · code · зусилля S · впевненість Medium

Місце	`backend/src/a11y/jira-bug.ts:43`
	`backend/src/a11y/jira-bug.ts:85-110`
	`backend/src/visual-bugs/visual-bug.ts:64-94`
	`backend/src/ui-tree/ui-tree.parser.ts:273-280`

Опис. Опис бага будується як Jira wiki markup. escapeWiki прибирає лише | і {} і застосовується тільки до рядків таблиці; `# ${v.message}` (містить describeNode — до 40 символів тексту елемента і повний resource-id з дерева), `* ${key}: ${value}` деталей і `Перевірка: ${check}` (назва перевірки з SDK) у visual-bug вставляються як є. Текст `[Увійти|https://evil.example]` або `!https://host/x.png!` стає посиланням/зовнішнім зображенням у баг-трекері банку. JQL і шляхи REST екрановані (encodeURIComponent, jira.client.ts:133-303; jqlFor бере лише код a11y-hex і asLabel).

Доказ. jira-bug.ts:43 `const escapeWiki = (value: string) => value.replace(/\|/g, '/').replace(/[{}]/g, '');`, :86 `...violations.map((v) => # ${v.message})`; visual-bug.ts:91 `Перевірка: ${check}`.

Вплив. Фішингові посилання або трекінг-зображення в баг-репортах, що їх створює довірений користувач від свого імені; вимагає контролю над текстами застосунку/назвою перевірки і дії людини.

Рекомендація. Екранувати спецсимволи wiki-розмітки ([ ] ! { } | * _ # h1.) у всіх рядках з даних (або загортати у {noformat}); для Jira Cloud — ADF з текстовими вузлами.

Low **VIS-SEC-055** · Нова **VLM-вердикт при прийомі запускає автоапрув і множить виклики Gemini без ліміту**

CWE / OWASP	CWE-1039 · A04:2021-Insecure Design
CVSS	`CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:L/A:N (4.2)`
Компонент / тип	backend · code · зусилля S · впевненість High

Місце	`backend/src/test-runs/test-runs.service.ts:770-850`
	`backend/src/compare/libs/vlm/vlm.service.ts:57-104`
	`backend/src/test-runs/test-runs.service.ts:661-678`

Опис. Для проєкту з рушієм vlm tryAutoApproveByNewBaselines при прийомі знімка викликає compareService.getDiff для кожного approved run з тим самим еталоном (цикл до першого збігу); кожен виклик — окремий запит у Gemini з трьома повними PNG і окремий diff-файл (VlmService примусово ставить saveDiffAsFile:true, :58-63, хоча shouldAutoApprove просить false). Якщо модель відповіла identical:true, знімок автоматично затверджується. Перепорівняння після зміни правила також шле в Gemini до MAX_RECOMPARE_AFTER_RULE=200 знімків фоново (:870). Денного ліміту чи обліку VLM-викликів немає (AI_DAILY_CALL_LIMIT — лише для AiService).

Доказ. test-runs.service.ts:804-818 findMany approved runs → :820-821 (і аналогічно :783-784) `if (await this.shouldAutoApprove(...)) return this.approve(testRun.id, false, true);`; :837 `const diffResult = await this.compareService.getDiff({...saveDiffAsFile: false})`; :848 `if (diffResult.status === TestStatus.ok) return true`; vlm.service.ts:58-63 `saveDiffAsFile: true`, : 98-99 `if (pass) result.status = TestStatus.ok`.

Вплив. Обсяг даних банку в Gemini на один знімок більший, ніж очевидно з налаштувань (вплив на умови контракту, VIS-SEC-003). Текст на екрані (prompt injection) або помилка моделі може не лише сховати зміну, а й автоматично затвердити її в гілці без людини. Накопичуються зайві diff-файли.

Рекомендація. Не дозволяти вердикту VLM вести до approve (автоапрув — лише за детермінованим порівнянням); рахувати VLM-виклики в ліміті інстансу; не зберігати diff у shouldAutoApprove. Зміна поведінки — через /spec.

Пов’язано VIS-SEC-003, VIS-SEC-032

Info

Info VIS-SEC-056 · Нова **StripUnknownFieldsPipe не чистить вкладені DTO: зайві ключі областей ігнорування зберігаються в JSON**

CWE / OWASP	CWE-915 · A08:2021-Software and Data Integrity Failures
CVSS	`n/a (Info)`
Компонент / тип	backend · code · зусилля S · впевненість High

Місце	`backend/src/shared/strip-unknown-fields.pipe.ts:33`
	`backend/src/test-runs/dto/update-ignore-area.dto.ts:11-14`
	`backend/src/test-runs/dto/create-test-request.dto.ts:38-43`
	`backend/src/test-runs/test-runs.service.ts:459`
	`backend/src/test-runs/test-runs.service.ts:725`

Опис. Пайп фільтрує лише ключі верхнього рівня; ValidationPipe без transform повертає вихідний plain-об'єкт, тож елементи `ignoreAreas` (IgnoreAreaDto — x, y, width, height) зберігають будь-які додаткові ключі. Вони потрапляють у `JSON.stringify(ignoreAreas)` → TestRun.ignoreAreas/ tempIgnoreAreas і TestVariation.ignoreAreas і віддаються клієнтам (у т.ч. через socket.io testRun_updated). Вкладених записів Prisma це не дає (поле — String), marks у visual-bugs нормалізуються (visual-bugs.service.ts:157).

Доказ. strip-unknown-fields.pipe.ts:33: `return Object.fromEntries(Object.entries(value).filter(([key]) => declared.has(key)));` — без рекурсії; test-runs.service.ts:725: `ignoreAreas: JSON.stringify(ignoreAreas)`.

Вплив. Немає ескалації чи обходу ізоляції; лише сміттєві дані в межах власного проєкту editor (обмежені лімітом тіла JSON) і ризик, що майбутній код почне передавати вкладений DTO у Prisma, покладаючись на пайп.

Рекомендація. Перед збереженням мапити області на {x,y,width,height} (як normalizeMarks) або зробити пайп рекурсивним для властивостей з @ValidateNested/@Type. Задокументувати в backend/AGENTS.md, що пайп діє лише на верхньому рівні.

Пов'язано VIS-SEC-012

Info VIS-SEC-057 · Нова **Застарілі облікові дані й чужа IP-адреса в документації та тестах**

CWE / OWASP	CWE-1188 · A05:2021-Security Misconfiguration
CVSS	`CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:N/I:N/A:N (0.0)`
Компонент / тип	infra · process · зусилля S · впевненість High

Місце	`DEPLOY.md:640-642`
	`sdk-java/README.md:89`
	`sdk-java/src/test/resources/vision_config_example.json:2`
	`sdk-java/src/test/java/ua/vision/sdk/VisionConfigTest.java:52`

Опис. DEPLOY.md радить входити `visual-regression-tracker@example.com / 123456`, хоча seed/ compose вже створюють admin@vision.local (docker-compose.demo.yml:131-132) — розбіжність підштовхує лишати дефолтний пароль і несе стару назву бренду. У SDK (README і тести) — публічна IP 162.243.161.172 з upstream як приклад apiUrl.

Доказ. DEPLOY.md:642 «если не заданы — `visual-regression-tracker@example.com` / `123456`»; sdk-java/README.md:89 `"apiUrl": "[http://162.243.161.172:4200](http://localhost:4200)"`.

Вплив. Плутанина при відновленні, пароль 123456 як «нормальний» крок; копіювання прикладу з чужою IP шле знімки на сторонній хост.

Рекомендація. Виправити DEPLOY.md (admin@vision.local, вимога змінити пароль); замінити IP на localhost/example у README і тестах SDK.

Пов'язано VIS-SEC-010

6.2. Знахідки 2026-11-05, що змінили стан або severity

Закрито **VIS-SEC-001** · Закрито · раніше Critical · S-7 **Масове присвоєння в PATCH /builds/:id: ескалація editor→admin і крадіжка даних між командами**

CWE / OWASP	CWE-915 · A08:2021-Software and Data Integrity Failures
CVSS	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H (9.6)`
Компонент / тип	backend · code · зусилля S · впевненість High

Місце	`backend/src/builds/dto/build-modify.dto.ts:1`
	`backend/src/builds/builds.service.ts:110`
	`backend/src/builds/builds.controller.ts:116`
	`backend/src/main.ts:42`

Перевірка 2026-11-07. backend/src/builds/dto/build-modify.dto.ts:4-30 — ModifyBuildDto тепер клас з @IsOptional/@IsString/@IsBoolean, buildChanges() бере лише ciBuildId (string) і isRunning (boolean); builds.controller.ts:113-119 (MixedGuard, RoleGuard, admin/editor; buildChanges у контролері) і builds.service.ts:108-112 (`data: buildChanges(modifyBuildDto)` — повторний вибір у сервісі). Тести: builds.controller.spec.ts:131, builds.service.spec.ts:448 (`user: { connect }` відкидається), strip-unknown-fields.pipe.spec.ts:34-38. Пошук `data: dto|...dto|data: body|...body` по backend/src (крім spec) не знайшов жодного update/updateMany/upsert/create, що отримує тіло запиту цілком: усі 46 викликів Prisma на запис мають явний перелік полів (projects.service.ts:128-139,194-210; users.service.ts:60,192; dbusers.service.ts:28,36; test-runs.service.ts:742-748 (лише comment); test-variations.service.ts:69-80; reports.service.ts:45; visual-bugs.service.ts:221; a11y-issues.service.ts:325 `...entry` — entry збирається в коді, не з тіла). Стенд (repro-004): PATCH /builds з branchName/number/user.update.role/projectId → поля ігноруються, роль alice лишається editor, bob отримує 404.

Що лишилось. Обходу не знайдено. Залишкова зміна поведінки: `{"isRunning":"false"}` (рядок) тепер 400 від ValidationPipe, а не запис — для випущених SDK (булеві значення або порожнє тіло) не впливає. ciBuildId editor може змінити лише в межах свого проєкту (унікальність projectId_ciBuildId).

Суть знахідки (2026-11-05). `ModifyBuildDto` — це TypeScript-інтерфейс без валідаторів, а глобальний `ValidationPipe` у main.ts:42 створений без `whitelist`. Тіло запиту повністю йде в `prisma.build.update({ data: modifyBuildDto })`. Через вкладені записи Prisma (`user.connect`/`user.update`, `projectId`) власник API-ключа будь-якої команди (роль editor) переписує чужі рядки.

Закрито **VIS-SEC-007** · Закрито · раніше High **Вхід через LDAP з порожнім паролем не відхиляється (ризик unauthenticated bind)**

CWE / OWASP	CWE-287 · A07:2021-Identification and Authentication Failures
CVSS	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N (8.1)`
Компонент / тип	backend · code · зусилля S · впевненість Medium
Місце	`backend/src/users/ldap/ldapusers.service.ts:141`

Перевірка 2026-11-07. backend/src/users/ldap/ldapusers.service.ts:162-164 `if (!userLoginRequestDto.password?.trim()) throw …` до будь-якого bind; тест ldapusers.service.spec.ts: 119-126 (it.each(['', ' ']) — bind не викликається).

Що лишилось. Обходу не знайдено: String.prototype.trim прибирає й Unicode-пробіли (NBSP, U+2028, BOM); символ нульової ширини (U+200B) лишає пароль непорожнім — це вже автентифікований bind, а не unauthenticated (RFC 4513 §5.1.2 — лише нульова довжина). Тип гарантує @IsString (user-login-request.dto.ts). Варто додатково вимкнути unauthenticated bind на боці каталогу.

Суть знахідки (2026-11-05). DTO входу перевіряє лише `@IsString()`; далі `bind(dn, '')` без перевірки непорожності пароля. Багато LDAP/AD за замовчуванням трактують bind із порожнім паролем як успішний анонімний bind.

Закрито **VIS-SEC-012** · Закрито · раніше Medium · S-7 **Глобальний ValidationPipe без whitelist/forbidNonWhitelisted/transform (системний корінь mass assignment)**

CWE / OWASP	CWE-915 · A08:2021-Software and Data Integrity Failures
CVSS	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:H/A:N (6.5)`
Компонент / тип	backend · code · зусилля S · впевненість High
Місце	`backend/src/main.ts:42`

Перевірка 2026-11-07. backend/src/shared/strip-unknown-fields.pipe.ts:26-38 — StripUnknownFieldsPipe після ValidationPipe (globalPipes(), main.ts:44; e2e використовують той самий globalPipes). Лише metadata.type==='body' і metadata.data===undefined (рядок 29); поля — з class-validator metadata, з батьками (рядки 8-14; OmitType/PickType з @nestjs/swagger копіюють метадані). Object.fromEntries над Object.entries: ключ `__proto__` із JSON.parse — власна властивість, відкидається фільтром, а fromEntries створює дані через CreateDataProperty — prototype pollution немає; class-transformer 0.5.1 (lock) без відомої PP. Порядок: глобальні пайпи → пайп методу (multipart: ValidationPipe({transform:true}) у test-runs.controller.ts:390) — strip іде до transform, тест strip-unknown-fields.pipe.spec.ts:63-86 і http.spec.ts:63. Тест маршрутів pipe.spec.ts: 94-142 + project-access.routes.spec.ts:88-95 (кількість *.controller.ts = кількість відомих контролерів) не дає новому контролеру пройти непоміченим. Перевірені руками маршрути (spec: 111-119): BackupsController.connect/saveSettings — клас admin-only (backups.controller.ts:14-15), saveSettings через checkSettings (backups.service.ts:263-272); ReportsController.create — sectionsOf/ languageOf з білого списку (reports/report-links.ts:17-23), пишуться явні поля (reports.service.ts: 45-54); TestRuns approve/reject/delete — ids йдуть лише як `where: { id }` у findUnique/update; нерядкові елементи інтерцептор пропускає (project-access.ts:49-50), але Prisma відхиляє не-string для унікального id до будь-якого запису; UsersController.delete — admin-only, `user.delete({ where: { id } })`. Query/param: єдиний `@Query()` цілим об'єктом — MergeParams (test-variations.controller.ts:54, types.ts:3-10), поля деструктуруються явно. Стенд (repro-004): зайві поля в PATCH /builds відкидаються; юніт-тести strip-unknown-fields.*.spec.ts зелені.

Що лишилось. Свідомі межі, не обхід: (1) лише верхній рівень — вкладені DTO (@ValidateNested: UpdateIgnoreAreasDto.ignoreAreas, CreateTestRequestDto.ignoreAreas) зберігають невідомі ключі; вони потрапляють лише в JSON.stringify(...) (test-runs.service.ts:459,725), не у вкладені записи Prisma — див. NEW-B-2 (Info). (2) Тіла без класу (масиви id, inline-типи) не фільтруються — покриті списком CHECKED_BY_HAND; ручна перевірка підтверджує, що вони безпечні. (3) Захист тримається на тому, що сервіси самі перелічують поля (як і раніше) — пайп є другим шаром. forbidNonWhitelisted свідомо не ввімкнено (правило 5, сумісність SDK).

Суть знахідки (2026-11-05). `new ValidationPipe()` без опцій: невідомі поля тіла не відсікаються в усьому API. Захист від mass assignment тримається лише на ручному виборі полів у кожному сервісі; VIS-SEC-001 — відомий інстанс, але ризик системний.

Medium **VIS-SEC-004** · Частково · раніше High **Обчислювальна DoS: аналіз доступності UI-дерева блокує event loop для всіх команд**

CWE / OWASP	CWE-400 · A05:2021-Security Misconfiguration
CVSS	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:N/I:N/A:H (7.7)`
Компонент / тип	backend · code · зусилля M · впевненість High

Місце	`backend/src/test-runs/test-runs.service.ts:144`
	`backend/src/ui-tree/ui-tree.service.ts:352`
	`backend/src/a11y/a11y.rules.ts:238-241`
	`backend/src/a11y/contrast.ts:26,49`

Перевірка 2026-11-07. backend/src/ui-tree/ui-tree.service.ts:257-260 (violations → diffWorkerPool.run({kind:'a11y'})); backend/src/a11y/a11y.job.ts:23-25; backend/src/compare/libs/pixelmatch/worker-job.ts:14; тест backend/src/a11y/a11y.job.spec.ts:11. Правила доступності справді у воркері під тайм-аутом 60 с. Знижено до Medium: правила a11y виконуються в пулі (a11y.job.ts), API не блокують; залишок у головному потоці винесено в VIS-SEC-046, виснаження пулу 8 одночасними 60-с завданнями зупиняє лише порівняння.

Що лишилось. (1) Решта роботи прийому з деревом лишилась у головному потоці: розбір XML (ui-tree.service.ts:276-290, parseUiTree), JSON.stringify+gzipSync дерева (293-298), декодування PNG знімка, еталона і еталона іншого пристрою (analyze → decode → hdd.service.ts:53 readPng, ui-tree.service.ts:337-349), diffTrees/componentChanges (381-391) — див. NEW-C-2. (2) GET /test-runs/:id/structure будує pickableElements з квадратичною складністю в головному потоці — NEW-C-1 (гірше за вихідну 004). (3) Велике дерево і декодований RGBA (до 160 МБ на 40 Мпкс) копіюються в воркер structured clone без transferList (diff-worker-pool.ts:178), по два завдання на analyze; черга (256) рахує завдання, а не байти — сотні МБ на завдання в пам'яті головного потоку. (4) Пул можна виснажити: size ≤ 8 воркерів (diff-worker-pool.ts:39-42), кожне важке завдання тримає воркер до 60 с; 8 паралельних завантажень на хвилину тримають пул зайнятим, далі 'Image diff queue is full' для всіх команд (68-70). (5) Перехід на головний потік після 3 падінь поспіль — NEW-C-4.

Суть знахідки (2026-11-05). На шляху прийому `analyzeUiTree` синхронно викликає `findViolations`, яка для кожного текстового вузла рахує контраст, семплюючи до MAX_SAMPLES=40000 пікселів (contrast.ts). Усе синхронно в головному потоці, без воркера. Дерево обмежене лише 10 МиБ після розпакування — це десятки тисяч вузлів.

Рекомендація. Винести аналіз доступності у воркер-пул із тайм-аутом; обмежити кількість вузлів дерева і сумарну площу контрастних боксів; відхиляти дерева понад розумну кількість вузлів. Потребує специфікації (зміна поведінки прийому).

Medium **VIS-SEC-005** · Частково · раніше High **Області ігнорування без меж координат зависають воркер порівняння без тайм-ауту**

CWE / OWASP	CWE-400 · A05:2021-Security Misconfiguration
CVSS	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:N/I:N/A:H (7.7)`
Компонент / тип	backend · code · зусилля S · впевненість High

Місце	`backend/src/compare/utils/index.ts:14-26`
	`backend/src/test-runs/dto/ignore-area.dto.ts:5-19`
	`backend/src/compare/diff-worker-pool.ts:52-70`

Перевірка 2026-11-07. backend/src/compare/utils/index.ts:18-19 (Math.max(0, area.y/x), межі зображення); backend/src/compare/diff-worker-pool.ts:46,177,185-196 (тайм-аут 60 с, воркер замінюється, без повтору, не рахується як spawn failure); тести backend/src/compare/utils/index.spec.ts, backend/src/compare/diff-worker-pool.spec.ts:86. Таймер знімається в message/replace (96, 127), terminated-воркер не повертається в idle (99, 122) — витоку воркерів не видно.

Що лишилось. Кількість областей не обмежена: CreateTestRequestDto.ignoreAreas — лише @IsArray/@ValidateNested (backend/src/test-runs/dto/create-test-request.dto.ts:38-43), UpdateIgnoreAreasDto без ArrayMaxSize (update-ignore-area.dto.ts:11-14), IgnoreAreaDto лише @IsNumber. N областей на весь кадр = N×W×H ітерацій на кожне з двох зображень (index.ts:15-26), аналогічно coveredPixels/applyTolerance (pixelmatch.core.ts:97-160). Тисячі областей у JSON до BODY_PARSER_JSON_LIMIT (20 МБ у демо) гарантують завдання на всі 60 с; збережені на варіації області роблять так кожне наступне порівняння. Зависання «назавжди» закрито, лишилась дешева DoS пулу на 60 с за запит.

Суть знахідки (2026-11-05). `applyIgnoreAreas` ітерує від `area.y/area.x` без нижньої межі; `IgnoreAreaDto` перевіряє лише `@IsNumber`. Величезні від'ємні координати з великими width/height дають майже нескінченний цикл. Пул воркерів порівняння не має тайм-ауту завдання. Область зберігається й застосовується до кожного наступного знімка перевірки.

Рекомендація. Обмежувати координати межами зображення при валідації і в applyIgnoreAreas (Math.max(0,...)); тайм-аут на завдання воркера. Потребує специфікації.

Low **VIS-SEC-006** · Частково · раніше High **Режим LDAP: будь-який користувач переприв’язує свій акаунт до чужої особи з каталогу**

CWE / OWASP	CWE-287 · A07:2021-Identification and Authentication Failures
CVSS	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N (9.3)`
Компонент / тип	backend · code · зусилля M · впевненість Medium
Місце	`backend/src/users/ldap/ldapusers.service.ts:113-121`

Перевірка 2026-11-07. backend/src/users/ldap/ldapusers.service.ts:130-151 (email з тіла, що відрізняється від current.email, -> 400; імена людей каталогу беруться з каталогу); тест backend/src/users/ldap/ldapusers.service.spec.ts:128-158 («does not let anyone take a colleague's email…», «keeps the email…», «lets a local account … rename itself»). Інших шляхів змінити email у режимі LDAP немає: UsersService.update делегує в LdapUsersService (users.service.ts:73-75), адмінського ендпоінта зміни email немає (users.controller.ts).

Що лишилось. (1) Прив'язка до акаунта досі за email, не за незмінним ідентифікатором каталогу (objectGUID/sAMAccountName): якщо банк повторно видасть адресу (новий співробітник з тим самим ім'ям), він увійде в старий акаунт звільненого (ldapusers.service.ts:179-183). (2) У режимі БД DbUsersService.update (dbusers.service.ts:35-46) і далі записує будь-який email з тіла — при переході БД→LDAP/SSO заздалегідь «зайнята» адреса колеги ловить його перший корпоративний вхід; також дає попереднє карбування JWT (NEW-A-3). (3) Наслідок самого виправлення: isInDirectory=false для невідомих відкрив локальний вхід для людей, яких більше немає в каталозі (NEW-A-1).

Суть знахідки (2026-11-05). `LdapUsersService.update` бере email з тіла PUT /users і записує в акаунт того, хто викликає, дані знайденої в каталозі людини. Нападник ставить собі email колеги; коли справжня людина вперше входить через LDAP, вона потрапляє в акаунт нападника.

Рекомендація. Не дозволяти змінювати email у LDAP-режимі; прив'язка лише за незмінним ідентифікатором каталогу (objectGUID/sAMAccountName), не за email із тіла запиту. Потребує специфікації.

Medium **VIS-SEC-014** · Частково · раніше Medium · S-1 **Немає меж розміру файла й розмірів PNG: multer без limits, повне декодування в головному потоці**

CWE / OWASP	CWE-400 · A05:2021-Security Misconfiguration
CVSS	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:H (6.5)`
Компонент / тип	backend · code · зусилля S · впевненість Medium
Місце	`backend/src/test-runs/test-runs.controller.ts:374` `backend/src/static/hdd/hdd.service.ts:48-55`

Перевірка 2026-11-07. backend/src/shared/screenshot-limits.ts:31-35 (сигнатура + 'IHDR' на зсуві 12), 44-61 (413/400), 63-75 (readPng); backend/src/test-runs/test-runs.service.ts:93-98; backend/src/test-runs/test-runs.controller.ts:367 (base64 → той самий screenshotProblem у сервісі), 386 (multer fileSize, files:1); ollama.controller.ts:54; hdd.service.ts:53, s3.service.ts:55, pixelmatch.core.ts:175-176, signature.core.ts:70-71 через readPng. Тести backend/src/shared/screenshot-limits.spec.ts:16-57. JSON-шлях обмежений body-parser (main.ts:60-62; Nest не додає свій jsonParser, коли middleware з таким ім'ям уже є). width*height — Number, переповнення немає. Стенд (repro-004): 30000×30000 і не-PNG відхиляються (413/400) у multipart і JSON; але обхід другим IHDR підтверджено наживо — див. VIS-SEC-045.

Що лишилось. Перевіряється лише перший IHDR; декодер (pngjs PNG.sync.read) потім бачить інше: повторний IHDR, черезрядкове (interlace) зображення з бомбою в IDAT, 16-бітна глибина, і канва max(w)×max(h) при allowDiffDimensions — див. NEW-C-3. Декодування в головному потоці (hdd.service.ts:53 з ui-tree.service.ts:321-327) лишилось.

Суть знахідки (2026-11-05). `FileInterceptor('image')` без `limits`; вміст не перевіряється на PNG; розміри з IHDR не обмежуються, декодування повне (pngjs) у головному потоці. Малий файл із великими заявленими розмірами (напр. 30000×30000) розпухає в пам'яті.

Рекомендація. `FileInterceptor('image',{limits:{fileSize}})`; перевірка сигнатури PNG; межа ширини×висоти до декодування; декодувати у воркері.

Low **VIS-SEC-036** · Частково · раніше Low **PUT /users записує email без нормалізації й перевірки дублікатів в іншому регістрі**

CWE / OWASP	CWE-706 · A04:2021-Insecure Design
CVSS	`n/a (Low)`
Компонент / тип	backend · code · зусилля S · впевненість High
Місце	`backend/src/users/dbusers.service.ts:35`

Перевірка 2026-11-07. Виправлено: users.service.ts:51-59 (createAccount — accountEmail() і перевірка mode:'insensitive'), ldapusers.service.ts:105,179 і oidcusers.service.ts:184 — lowercase. Не виправлено: backend/src/users/db/dbusers.service.ts:35-43 (PUT /users у режимі БД пише email як є, без trim/lowercase і без перевірки дубліката без регістру); dbusers.service.ts:59-61 (вхід — точний збіг без нормалізації).

Що лишилось. У режимі БД можна мати «Ann@bank.ua» поруч з «ann@bank.ua»; людина з великою літерою в email не входить (а спроби рахуються лімітом за lowercase-ключем). Унікального індексу без регістру немає.

Суть знахідки (2026-11-05). Email записується як є; два акаунти з різним регістром однакового email можливі — плутанина в журналі рішень і при вході.

Рекомендація. Нормалізувати email (lowercase) і забезпечити унікальність без урахування регістру (додавальний індекс).

Medium **VIS-SEC-035** · Не виправлено · раніше Low **Звільнені з каталогу/IdP зберігають доступ через API-ключ і локальний вхід**

CWE / OWASP	CWE-613 · A07:2021-Identification and Authentication Failures
CVSS	`n/a (Low)`
Компонент / тип	backend · code · зусилля S · впевненість High
Місце	`backend/src/users`

Перевірка 2026-11-07. backend/src/auth/guards/api.guard.ts:10-21 і role.guard.ts:164-177 — ключ без строку дії, без перевірки isActive/каталогу; users.service.ts:123-126 — людина, якої немає в каталозі, входить локальним паролем.

Що лишилось. Погіршено виправленням 41b15e0: раніше для відсутньої в каталозі людини findUserInLdap кидав 400 і вхід не проходив; тепер isInDirectory=false переводить її на локальний пароль — зокрема скопійований з VRT або встановлений самою людиною живим токеном. Окремо — NEW-A-1 (High).

Суть знахідки (2026-11-05). API-ключ не має строку дії й не перевіряє стан акаунта в каталозі; після вимкнення в LDAP/IdP доступ через ключ (і локальний пароль, якщо заданий) лишається.

Рекомендація. Перевіряти стан акаунта при кожному запиті; строк дії й ротація ключів.

High **VIS-SEC-003** · Прийнятий ризик · раніше High **VLM-рушій автоматично надсилає знімки банку в Gemini при прийомі (напруга з правилом 6 / ADR-0007)**

CWE / OWASP	CWE-200 · A01:2021-Broken Access Control
CVSS	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:L/A:N (8.5)`
Компонент / тип	backend · code · зусилля M · впевненість High

Місце	`backend/src/test-runs/test-runs.service.ts:140-150`
	`backend/src/compare/compare.service.ts:62-63`
	`backend/src/compare/libs/vlm/vlm.service.ts:90-99`
	`backend/src/ai/gemini.service.ts:44`
	`docs/specs/035-admin-accounts/spec.md:192-193`

Перевірка 2026-11-07. Що йде в Gemini: три повні PNG — еталон і новий знімок (з замальованими ignore-областями) та diff (backend/src/compare/libs/vlm/vlm.service.ts:76-90), промпт із налаштувань проєкту і модель (gemini.service.ts:42-52), ключ — з Project.imageComparisonConfig, розшифровується лише тут (vlm.service.ts:51-54). Коли: лише якщо project.imageComparison=vlm I config.provider='gemini' I pixelmatch дав не ok/new (vlm.service.ts:66-72). Точки виклику: прийом POST /test-runs → calculateDiff (test-runs.service.ts:343-357); автоапрув за новими еталонами — по одному getDiff на кожен approved run гілки (test-runs.service.ts:798-822, 837); перепорівняння після зміни правила/ignore-областей (test-runs.service.ts:661-678, до MAX_RECOMPARE_AFTER_RULE). Вимикач: немає ні на інстанс (жодної змінної; compare.service.ts:45-64 повертає VlmService без перевірок; AI_PROVIDER читає лише ai.service.ts:53-58), ні окремого на проєкт, крім зміни рушія editor-ом. Правило 6 поза VLM: AiFeaturesService/AiService викликаються лише з ai.controller.ts: 96-119 (на вимогу); у test-runs/, ui-tree/, a11y/ викликів AI немає (grep AiService|GeminiService| OllamaService).

Що лишилось. Прийнятий ризик (контракт з Gemini). Лишається: (1) немає вимикача рівня інстансу на випадок відкликання погодження; (2) немає денного ліміту/обліку VLM-викликів (AI_DAILY_CALL_LIMIT стосується лише AiService), а автоапрув і перепорівняння множать виклики на один знімок; (3) вердикт моделі сам ставить ok і може запустити автоапрув (NEW-D-1).

Суть знахідки (2026-11-05). Якщо проєкт обрав рушій порівняння `vlm`, прийом знімка (POST / test-runs → calculateDiff → compareService.getDiff → getComparator → VlmService → GeminiService.generate) СИНХРОННО надсилає в Gemini три повні PNG (еталон, новий, diff). Вмикає будь-який editor проєкту; вимикача рівня інстансу немає, `AI_PROVIDER=none` на це не впливає. Вердикт моделі сам ставить статус (vlm.service.ts:98-99).

Рекомендація. Для банку: вимикач рівня інстансу (hard off, напр. VISION_VLM_ALLOWED=false), що перекриває вибір проєкту; явне погодження ІБ на кожен проєкт із VLM; ніколи не дозволяти вердикту моделі самому затверджувати. Зміна поведінки — потребує специфікації (/spec).

High **VIS-SEC-002** · Відкладено · раніше High · S-4 **Приймаються відомі/слабкі секрети підпису; DEPLOY.md радить секрет робочого VRT**

CWE / OWASP	CWE-1392 · A07:2021-Identification and Authentication Failures
CVSS	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:N (9.3)`
Компонент / тип	backend · config · зусилля S · впевненість High

Місце	`backend/src/main.ts:29-34`
	`backend/src/shared/secret-box.ts:43-53`
	`backend/src/shared/secret-box.spec.ts:25`
	`docker-compose.demo.yml`
	`DEPLOY.md:108`

Перевірка 2026-11-07. backend/src/main.ts:29-35 — перевіряється лише наявність VISION_SECRET_KEY (serverSecretKey, backend/src/shared/secret-box.ts:13-15,43-52: будь-який непорожній рядок → SHA-256); JWT_SECRET так само. .env.demo.example:16,19 `JWT_SECRET=change-me`, `VISION_SECRET_KEY=change-me`; backend/.env.example:2,5; DEPLOY.md:108 досі радить `JWT_SECRET` «можна той же, что у прода». Змін з 7b13af1 немає.

Що лишилось. Відкладено рішенням власника; без змін: слабкі/відомі секрети приймаються, порада взяти секрет проду в DEPLOY.md лишилась. Слабкий VISION_SECRET_KEY також робить розшифровуваним sealed apiKey Gemini у Project.imageComparisonConfig, що йде в бекап (VIS-SEC-013).

Суть знахідки (2026-11-05). Старт перевіряє лише, що `JWT_SECRET` і `VISION_SECRET_KEY` непорожні; значення `change-me` чи `x` приймаються (тест secret-box.spec.ts:25 це закріплює). Приклади середовища містять `change-me`. DEPLOY.md радить узяти секрет робочого VRT — тоді JWT Vision дійсні й на проді (порушення правила 1).

Рекомендація. Відмовлятися стартувати, якщо секрет у чорному списку відомих значень або коротший за N; вимагати різні секрети для Vision і VRT; прибрати пораду з DEPLOY.md. Потребує мінімальної зміни bootstrap.

Low **VIS-SEC-024** · Не виправлено · раніше Low · S-3 **Шифрування секретів без AAD, KDF = один SHA-256; приймається вже зашифроване значення**

CWE / OWASP	CWE-327 · A02:2021-Cryptographic Failures
CVSS	`n/a (Low)`
Компонент / тип	backend · code · зусилля S · впевненість High
Місце	`backend/src/shared/secret-box.ts:12-23,34`

Перевірка 2026-11-07. backend/src/shared/secret-box.ts:13-15 (KDF = SHA-256 рядка), :21-23 (GCM без AAD); backend/src/projects/project-secrets.ts:96-98 — значення з префіксом enc:v1: від клієнта зберігається як є. Semgrep (gcm-no-tag-length): secret-box.ts:34 — createDecipheriv без authTagLength, setAuthTag приймає й укорочений тег (4–16 байт); для знаючого запис у БД це спрощує підробку. Додати { authTagLength: 16 } і перевірку tag.length.

Що лишилось. Без змін.

Суть знахідки (2026-11-05). AES-256-GCM без AAD; ключ — простий SHA-256 від рядка без перевірки довжини/ентропії. `keepConfigSecrets` приймає від клієнта вже зашифроване значення з префіксом enc:v1: (можлива підстава чужого шифротексту).

Рекомендація. Додати AAD (контекст поля), вимагати достатню ентропію ключа, не приймати enc:-значення ззовні.

Low **VIS-SEC-032** · Не виправлено · раніше Low **Prompt injection: тексти екрана й назви перевірок у промптах; вердикт VLM сам ставить статус**

CWE / OWASP	CWE-94 · A03:2021-Injection
CVSS	`n/a (Low)`
Компонент / тип	backend · code · зусилля S · впевненість Medium
Місце	`backend/src/ai/gemini.service.ts` `backend/src/compare/libs/vlm/vlm.service.ts:98-99`

Перевірка 2026-11-07. backend/src/compare/libs/vlm/vlm.service.ts:98-102 — `identical:true` від моделі ставить TestStatus.ok; промпт і дані в одному запиті (gemini.service.ts:42). Додатково: test-runs.service.ts:828-850 — ok від VLM у shouldAutoApprove веде до `this.approve(testRun.id, false, true)` (:784 і :821 — два шляхи автоапруву).

Що лишилось. Вердикт моделі не лише ставить ok, а й автоматично затверджує знімок у гілці (NEW-D-1).

Суть знахідки (2026-11-05). Тексти з UI-дерева й назви тестів потрапляють у промпт чернетки бага і VLM-порівняння; зловмисний текст на екрані може спотворити чернетку або (для VLM) вплинути на вердикт, який сам ставить статус ok.

Рекомендація. Не дозволяти вердикту моделі самому затверджувати; розділяти інструкції й дані в промпті; екранувати.

Medium **VIS-SEC-008** · Не виправлено · раніше Medium · S-2 **Allowlist Jira перевіряє лише hostname: проходять http, будь-який порт і редиректи**

CWE / OWASP	CWE-918 · A10:2021-Server-Side Request Forgery
CVSS	`CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:C/C:H/I:L/A:N (6.8)`
Компонент / тип	backend · code · зусилля S · впевненість High
Місце	`backend/src/a11y/jira-hosts.ts:15-22` `backend/src/a11y/jira.client.ts:102`

Перевірка 2026-11-07. backend/src/a11y/jira-hosts.ts:80-87 — досі лише `new URL(url).hostname`; backend/src/a11y/jira.client.ts:101-105 — fetch без `redirect`, схема/порт не перевіряються. Змін у jira-hosts.ts немає з 7b13af1.

Що лишилось. Нова поверхня з #23–#31: GET /projects/:id/jira/types?url=… (projects.controller.ts: 63-73 → jira-access.ts:52-65) бере НЕЗБЕРЕЖЕНУ адресу з query і шле на неї особистий токен після тієї ж перевірки лише hostname: проходять http://дозволений-хост:будь-який-порт/будь-який-шлях. Відповідь не віддзеркалюється (лише 'failed'/типи, projects.service.ts:143-154), токен — власний, тож вплив як у 008. `@`-трюк закритий (hostname після @), DNS rebinding — у межах DNS банку. Node fetch прибирає Authorization при міжоріджинному редиректі.

Суть знахідки (2026-11-05). `isAllowedJiraUrl` звіряє лише `new URL(url).hostname`. Проходять `http://` (токен відкритим текстом), будь-який порт, інші схеми. `fetch` іде за редиректами; Node прибирає Authorization при зміні хоста, але тіло POST з вкладеннями-скриншотами при 307 пересилається, а частина відповіді внутрішнього сервісу повертається користувачу (SSRF).

Рекомендація. Вимагати https, звіряти host+port, заборонити редиректи (`redirect: 'error'`) або повторно валідувати кінцевий URL; заборонити приватні IP.

7. Дані за межами контуру та LLM

Повна таблиця потоків — `audit/raw/data-egress.md` (не змінилась).

VLM на Gemini при прийомі знімка (VIS-SEC-003) — прийнятий ризик. Рішення власника 2026-11-07: з Gemini є контракт. Що саме і коли виходить за межі контуру (перевірено за кодом `a3b6104`):

- **Що йде.** Три повні PNG: еталон, новий знімок (обидва з замальованими областями ігнорування) і diff. До них — промпт із налаштувань проєкту й ключ проєкту.
- **Коли.** Лише якщо водночас `imageComparison=vlm`, провайдер `gemini` і різниця пікселів понад допуск. Таке відбувається:
  - при прийомі (`test-runs.service.ts:357`);
  - в автоапруві — по одному виклику на кожен уже затверджений запуск гілки (`:783`, `:820-837`);
  - при перепорівнянні після зміни правила — до 200 знімків (VIS-SEC-055).
- **Вимикача рівня інстансу немає.** Немає й денного ліміту викликів. Рушій проєкту вмикає й вимикає editor.
- **Решта LLM лише на вимогу.** Окрім VLM-рушія, LLM-функції викликаються тільки з `ai.controller.ts`, тобто людиною. Правило 6 поза VLM дотримано.

Рекомендації в межах прийнятого ризику:

- **Перевірити, що договір покриває всі шляхи.** Потрібно, щоб контракт покривав і автоапрув, і перепорівняння, а не лише прийом.
- **Додати ліміт і облік.** Змінна рівня інстансу, щоб вимкнути VLM на випадок відкликання погодження, і денний ліміт VLM-викликів з обліком.
- **Обмежити особисті ключі.** Особисті ключі Gemini (ADR-0020) і `AI_PROVIDER=none` не узгоджені (VIS-SEC-016). Варто визначити, чи контракт поширюється й на особисті ключі.
- **Не давати VLM самій затверджувати знімки.** Вердикт VLM сам затверджує знімок у гілці (VIS-SEC-032, 055). Через prompt injection у тексті екрана модель може пропустити регресію.

Jira. Особистий токен іде лише на hostname з `VISION_JIRA_HOSTS`. Схема, порт і редиректи не перевіряються (VIS-SEC-008). На новому маршруті `jira/types` адресу задає editor у query, ще до збереження (VIS-SEC-048).

Google Drive. Бекап без шифрування (VIS-SEC-013, ADR-0012) — без змін.

8. Комерційне використання та ліцензії

Висновки розділу 8 звіту 2026-11-05 чинні. Повторна перевірка:

- **Атрибуція.** `LICENSE` і `NOTICE` є в корені й у пакетах. Бренду «Visual Regression Tracker» в UI, i18n і образах немає: grep після #23–#31 нових входжень не дав.
- **Чого бракує.** `LICENSE` і `NOTICE` досі не копіюються в Docker-образи та jar SDK. Текст ліцензії шрифтів `@fontsource` (OFL-1.1) не потрапляє в збірку фронтенду (VIS-SEC-039).
- **Ліцензії залежностей.**
  - Прод бекенду: 452 пакети. MIT 324, Apache-2.0 58, ISC 34, BSD 19, BlueOak 8, Unlicense 2, PSF 1, 0BSD 1.
  - Єдиний EPL-2.0 — `elkjs` (через `prisma → @prisma/studio-core`). Він вирізається при збірці образу, тож це false positive, як і раніше.
  - SBOM (syft): 805 компонентів. Copyleft у поставку не потрапляє.
- **Install-скрипти.** Скрипти залежностей `protobufjs` (postinstall) і `@google/genai` (preinstall) переглянуто: перший лише перевіряє схему версій, другий — no-op.
- **Чужі дані в дереві.**
  - Фото реальних людей у фікстурах (VIS-SEC-042) — без змін.
  - DEPLOY.md радить вхід upstream `visual-regression-tracker@example.com / 123456`.
  - У прикладах SDK — публічна IP 162.243.161.172 з upstream (VIS-SEC-057).
- **Персональні й банківські дані.** Контрольні питання для юриста й ІБ — як у звіті 2026-11-05. Нове: VIS-SEC-044 стосується керування доступом. Звільнений співробітник зберігає доступ до знімків, доки адмін не видалить акаунт вручну. Для банку це питання процедури звільнення та вимог НБУ до керування доступом.

9. Roadmap виправлень

[спец] — змінює поведінку, потрібна специфікація (`/spec`). **[міг]** — потрібна додавальна міграція (ADR-0003).

До перенесення в контур:

Знахідка	Дія	Обсяг
VIS-SEC-043	Ліміт вузлів (і сиблінгів) дерева при прийомі; `pickableElements` — з картою позицій за один прохід (O(n)) або у воркері з тайм-аутом	S/M
VIS-SEC-045	Відкидати PNG з кількома IHDR; ліміт пікселів перевіряти і в `readPng` після розбору заголовка pngjs (або `PNG.sync.read` з `checkCRC` і власним парсером чанків); обмежити канву `allowDiffDimensions`	S
VIS-SEC-044	Позначка походження акаунта (local / ldap / sso) **[міг][спец]**: у режимі SSO/LDAP форма пароля — лише для `local`; скопійованим з VRT акаунтам — скинути пароль	M
VIS-SEC-002	Відмова стартувати з відомим/коротким секретом; прибрати пораду про секрет VRT із DEPLOY.md	S
VIS-SEC-046	Розбір і серіалізація дерева — у воркер; ліміт глибини	M

30 днів:

- **Сесії й TLS:**
  - VIS-SEC-009 — відкликання сесій **[спец][міг]**;
  - VIS-SEC-011 — CSP, HSTS і TLS у поставці;
  - VIS-SEC-018 — `state` для `#token=`.
- **Jira і бекап:**
  - VIS-SEC-008, 048 — https, порт, без редиректів; `jira/types` лише для збереженої адреси;
  - VIS-SEC-013 — бекап без Google Drive або з шифруванням.
- **Ролі й адмінські дії:**
  - VIS-SEC-020 — SSO не підвищує понижену роль;
  - VIS-SEC-017 і аудит-лог дій адміна;
  - VIS-SEC-010 — пароль seed не в лог, примусова зміна.
- **Вузькі місця обробки й середовище:**
  - VIS-SEC-005 — ліміт кількості областей;
  - VIS-SEC-015 — RE2 або ліміт часу regex;
  - VIS-SEC-016, 019, 021, 035.

90 днів:

- решта Low та Info;
- для VIS-SEC-003: вимикач і ліміт;
- ротація секретів і процедура реагування;
- прогін образів сканером банку.

10. Додатки

10.1. Результати сканерів (2026-11-07)

Сканер	Результат	Розбір
npm audit (prod+dev; backend, backend/prisma, frontend)	0 / 0 / 0	збігається з 2026-11-05
osv-scanner	не виконано (api.osv.dev 403)	`audit/raw/osv-scanner.err.txt`; результат 2026-11-05 (0) лишено
semgrep (969 локальних правил)	1212 спрацювань, 24 правила	1017 `i18next-key-format`, 15 `jsx-not-internationalized`, 32 `react-props-spreading` — стиль; `path-join-resolve-traversal` / `detect-non-literal-fs-filename` — FP (тести; `hdd.service` перевіряє вихід за теку, підтверджено тестом; бекапи — сталі імена `FILES.*`); `detect-non-literal-regexp` — TP VIS-SEC-015; `no-replaceall` — TP VIS-SEC-050; `gcm-no-tag-length` — TP, додано до VIS-SEC-024; `dynamic-urllib` у CLI — VIS-SEC-028
bandit (CLI)	B310×2, B404, B603	B310 = VIS-SEC-028; subprocess — фіксовані аргументи
shellcheck	43 (2 error SC2148, 5 warning)	стиль; без нових знахідок
hadolint	DL3066×4 (info), DL3008 (warning)	дрібне
gitleaks (dir)	11	6 — самоспрацювання на `audit/raw/gitleaks.json` попереднього прогону; 5 — тестові фікстури й приклади README, як 2026-11-05. Усі FP
license-checker (backend prod)	усе пермісивне	EPL-2.0 `elkjs` — FP (вирізається при збірці)
jest (backend, юніт)	818/818	тести виправлень зелені

10.2. Відтворення на стенді

- `audit/raw/repro-004-fixes-recheck.txt` — перевірка виправлень VIS-SEC-001, 012, 014.
- `audit/raw/repro-005-new-dos.txt` — VIS-SEC-043, 045, 049, 050.

10.3. SBOM

`audit/sbom.cdx.json` (CycloneDX, syft 1.18.1, коміт `a3b6104`) — 805 компонентів.

10.4. Перевірені й не перевірені області

Перевірено:

- усі виправлення з `audit/PROGRESS.md`;

- усі записи Prisma на предмет передачі тіла запиту цілком (46 місць — чисто);
- правило 7 (рядок `User` не віддається, `jiraToken` не виходить);
- нові маршрути `jira/types` і `trend`, сторінка входу і UI #23–#31 (XSS, open redirect, токени в URL);
- socket.io;
- compose, Dockerfile, nginx;
- скрипти й CI;
- ліцензії.

Не перевірено: пакети ОС в образах; runtime-ізоляція й ліміти compose; живі LDAP/IdP; транзитивний граф Gradle; атаки розміром понад кілька секунд (за правилами промту).